

Laboratory Record

CSE3015 AWS Cloud Practitioner

A14+D11+D12, BL2026270100433,
Fall Semester :2026-2027

Name of the student: Shashi Singh
Register Number: 24BAI10686

Programme Outcomes (POs):

- Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.
- Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
- Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to professional engineering practice.
- Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for, sustainable development.
- Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
- Communication: Communicate effectively on complex engineering activities with the engineering community and society at large, including comprehending and writing effective reports and design documentation, making effective presentations, and giving and receiving clear instructions.
- Project management and finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects in multidisciplinary environments.
- Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

INDEX

Exp. No.	Date	Name of the Experiment	Page Number
1	06-09-2026	Procedure to Create an AWS Account and Working with Amazon S3	
2	06-09-2026	Create a Lambda Function and Configure Event Triggers	
3	06-09-2026	Identity and Access Management (IAM) in AWS	

Exp No: 1	Procedure to Create an AWS Account and Working with Amazon S3
06-09-2026	

Aim:

- To create an Amazon Web Services (AWS) account and explore the AWS Management Console.
- To create an S3 bucket, enable versioning, configure encryption, set a bucket policy, and perform file upload and download operations.

Objectives:

- To understand the process of creating and setting up an AWS account.
- To explore and navigate the AWS Management Console.
- To create and configure an Amazon S3 bucket with versioning and encryption.
- To apply bucket policies and perform object upload/download operations.
- To understand public access settings and their security implications.

Tools Used:

- Amazon Web Services (AWS) — aws.amazon.com
- Web Browser (Chrome / Firefox)
- AWS Management Console
- Amazon S3 (Simple Storage Service)

Theory:

Amazon Web Services (AWS) is a comprehensive cloud computing platform provided by Amazon. It offers a wide range of cloud services including computing power, storage, databases, networking, machine learning, and more. AWS allows individuals and organizations to access scalable infrastructure on a pay-as-you-go basis without upfront investment in physical hardware.

Amazon S3 (Simple Storage Service) is an object storage service offering industry-leading scalability, data availability, security, and performance. S3 allows users to store and retrieve any amount of data at any time. Data is stored as objects within buckets. Each object consists of a file and optionally any metadata describing that file.

Key S3 Concepts:

- Bucket: A container for storing objects (files). Bucket names must be globally unique.
- Object: A file stored in a bucket along with its metadata.
- Versioning: Enables keeping multiple versions of an object in the same bucket.
- Encryption (SSE): Server-Side Encryption protects data at rest using AWS-managed or KMS keys.
- Bucket Policy: A JSON-based access control policy that defines who can access your bucket.
- Regions: AWS data centers are distributed globally. Selecting a nearby region reduces latency.

Procedure:

Part A: Creating an AWS Account

Step 1: Visit AWS Official Website

- Go to: https://aws.amazon.com
- Click on "Create an AWS Account" (top-right corner).

Step 2: Enter Email & Set Root User Details

- Email address: Use a valid email (e.g., Gmail).
- Password: Create a strong password.
- AWS Account name: Enter your name or organization name.
- Click Continue.

Step 3: Choose Account Type

- Choose Personal (for students/individual use) or Professional (for company/organizational use).
- Enter full name, address, and mobile number.

Step 4: Payment Information

- Enter a credit/debit card (Visa/MasterCard/UPI-enabled cards work).
- ₹2 is deducted temporarily for card verification (refunded later).
- Click Verify and Add.

Step 5: Identity Verification

- Select SMS or voice call for phone verification.
- Enter the OTP (One Time Password) sent to your mobile.
- Once verified, click Continue.

Step 6: Select a Support Plan

- Choose Basic Support Plan (Free) — includes 12-month Free Tier (e.g., EC2, S3, Lambda, RDS have free usage limits).
- Click Complete Sign-Up.

Step 7: Sign In to AWS Console

- Go to https://console.aws.amazon.com
- Login using your email (root user) and password.
- You now have access to the AWS Management Console.

Note: Set up billing alerts to avoid unexpected charges.

Part B: Working with Amazon S3

Step 1: Create an S3 Bucket

- Sign in to AWS Console and go to S3 from the Services menu.
- Click Create bucket.
- Fill in:
  - Bucket name: Must be globally unique (e.g., my_s3bucket2025).
  - Region: Choose your preferred region (e.g., Asia Pacific (Mumbai)).
- Uncheck "Block all public access" only if you need public access (not recommended for sensitive data).
- Leave other settings default.
- Click Create bucket.

Issue 1: Region must be selected from the top bar of the console.

Issue 2: If "Block all public access" is unchecked, the bucket policy editor becomes accessible. Both approaches should be tried and documented with screenshots to understand the difference.

Step 2: Enable Bucket Versioning

- Go to your bucket → click the Properties tab.
- Scroll to Bucket Versioning → click Edit → Enable → Save changes.
- Versioning keeps multiple versions of an object (useful in case of accidental deletes or overwrites).

Step 3: Enable Server-Side Encryption

- In the Properties tab, scroll to Default encryption.
- Click Edit → Choose:
  - AWS Key (SSE-S3) — simple and free.
  - Or KMS (SSE-KMS) — more secure, supports audit logs.
- Click Save changes.

Step 4: Add a Bucket Policy (Optional: Allow Read Access)

- Go to Permissions tab → Bucket Policy → Edit.
- Paste the following example policy to make objects publicly readable:

```
{
 "Version": "2012-10-17",
 "Statement": [
   {
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my_s3bucket2025/*"
   }
 ]
}
```

Step 5: Upload an Object

- Open your bucket.
- Click Upload → Add Files (e.g., a PDF, image, or .txt file).
- Click Upload.

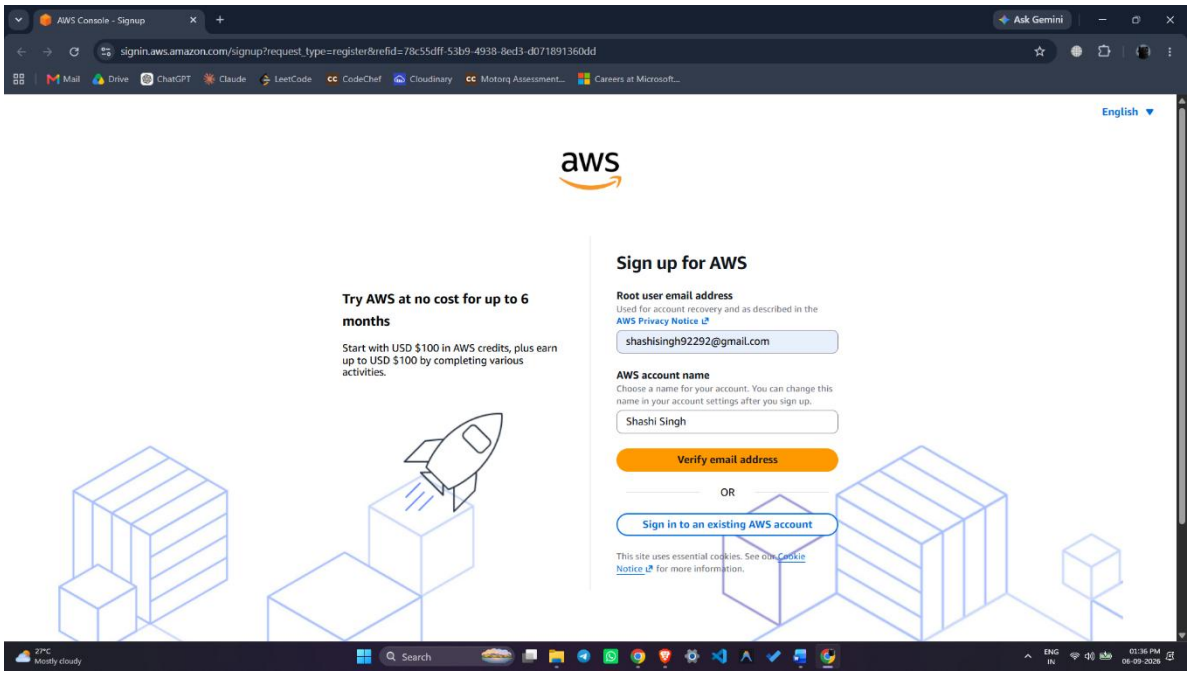
Issue: You can create a simple text file on your system and upload it if you do not have another file handy.

Step 6: Download an Object

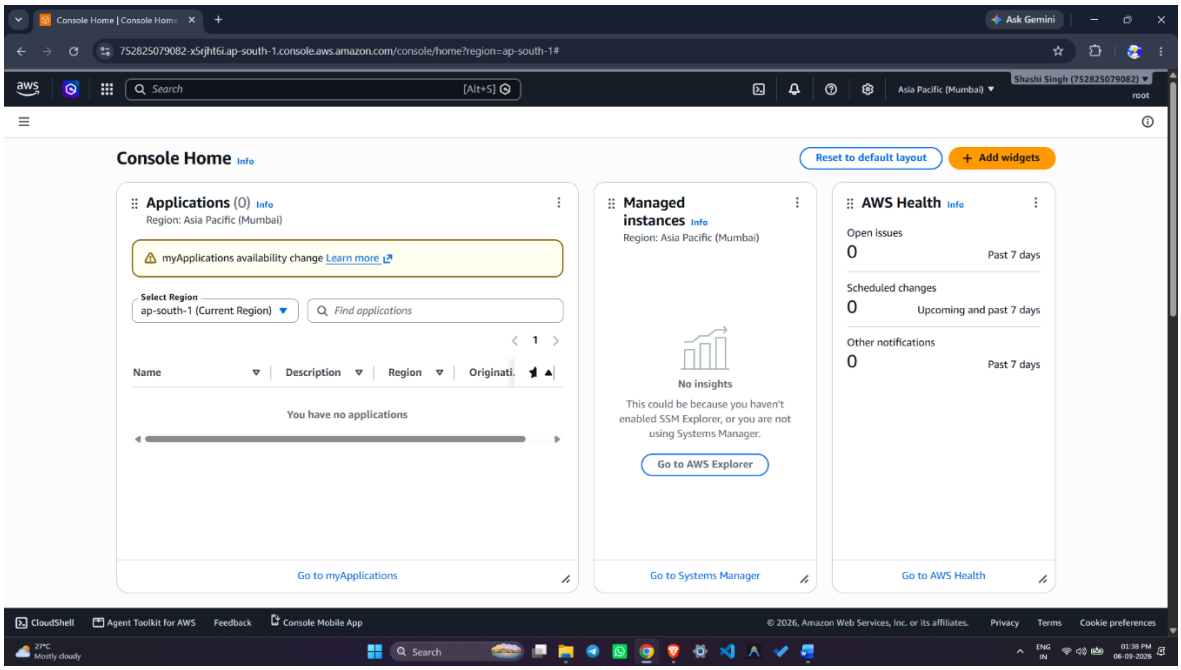
- Go to your bucket → Click on the object.
- Click Download.

Output:

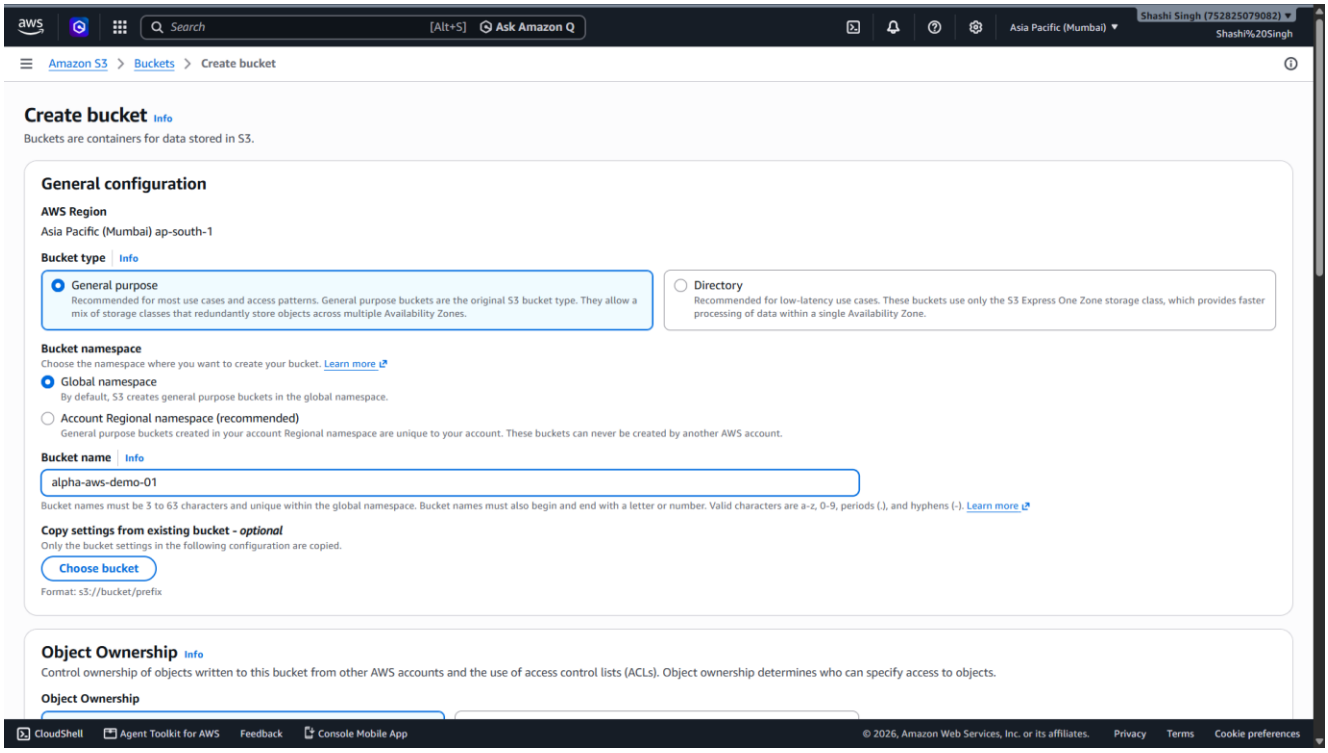
- Screenshot 1: AWS Account Creation — Sign-in page

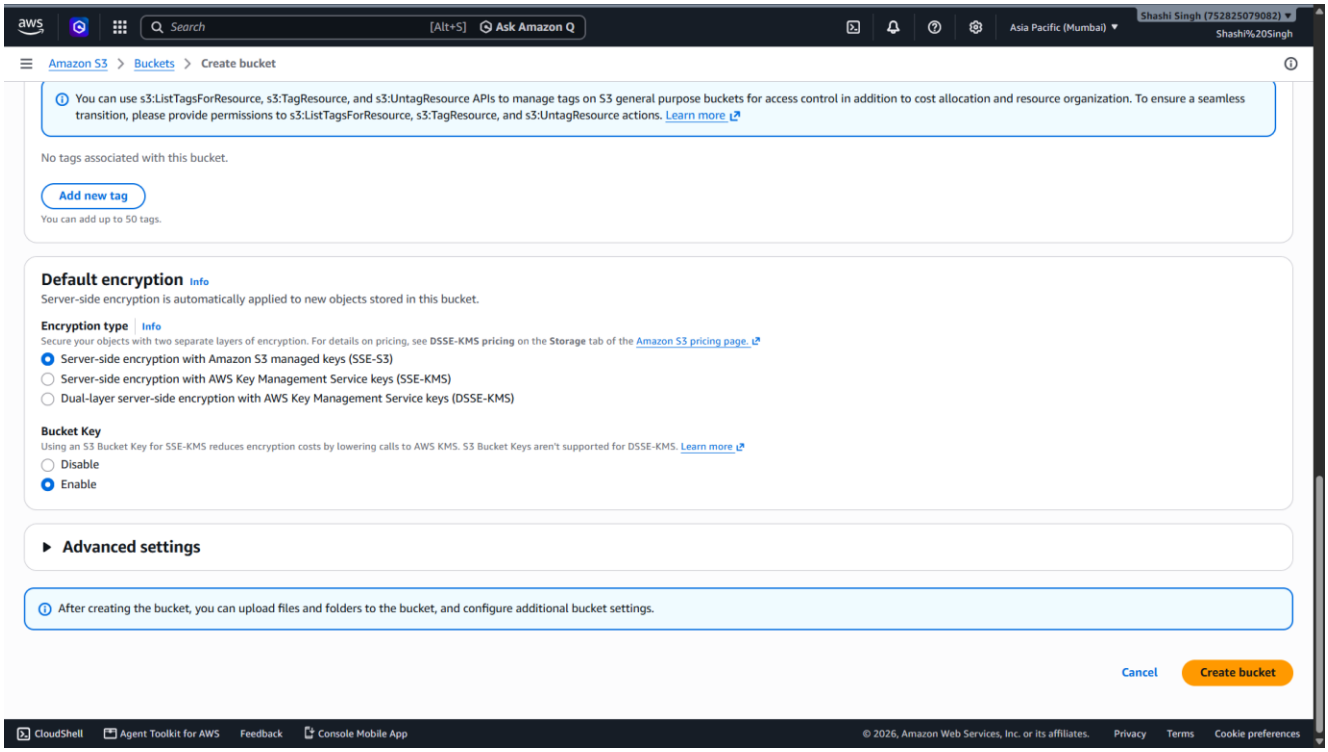


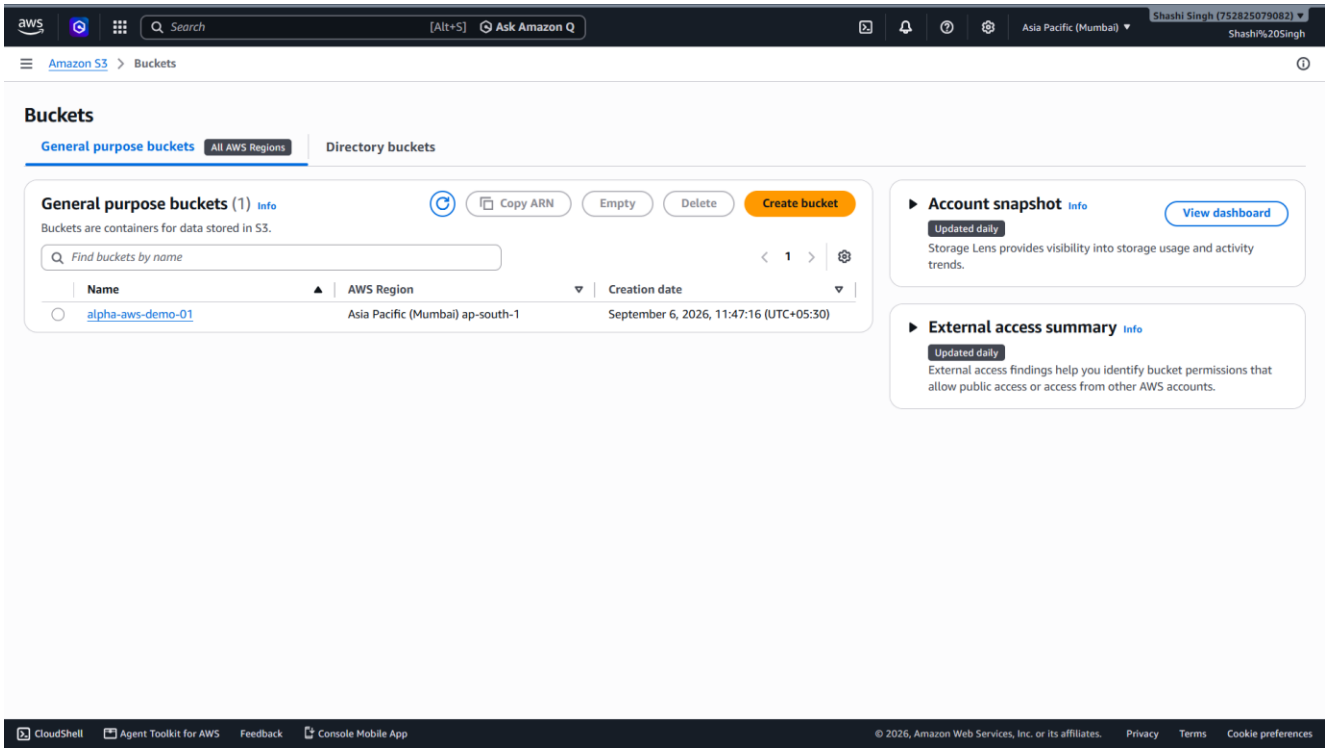
- Screenshot 2: AWS Management Console Dashboard



- Screenshot 3: S3 Bucket Created (name, region visible)




Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Shashi Singh (752825079082)
Shashi%20Singh
Amazon S3 > Buckets > Create bucket
You can use s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. Learn more
No tags associated with this bucket.
Add new tag
You can add up to 50 tags.
Default encryption Info
Server-side encryption is automatically applied to new objects stored in this bucket.
Encryption type Info
Secure your objects with two separate layers of encryption. For details on pricing, see DSSE-KMS pricing on the Storage tab of the Amazon S3 pricing page.
Server-side encryption with Amazon S3 managed keys (SSE-S3)
Server-side encryption with AWS Key Management Service keys (SSE-KMS)
Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)
Bucket Key
Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. Learn more
Disable
Enable
Advanced settings
After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.
Cancel
Create bucket
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Shashi Singh (752825079082)
Shashi%20Singh
Amazon S3 > Buckets
Buckets
General purpose buckets
All AWS Regions
Directory buckets
General purpose buckets (1) Info
Buckets are containers for data stored in S3.
Copy ARN
Empty
Delete
Create bucket
Find buckets by name
Name
AWS Region
Creation date
alpha-aws-demo-01
Asia Pacific (Mumbai) ap-south-1
September 6, 2026, 11:47:16 (UTC+05:30)
Account snapshot Info
View dashboard
Updated daily
Storage Lens provides visibility into storage usage and activity trends.
External access summary Info
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Result:

Successfully created an AWS account and logged into the AWS Management Console. An Amazon S3 bucket was created with a globally unique name in the Asia Pacific (Mumbai) region. Bucket versioning and server-side encryption (SSE-S3) were enabled. A bucket policy was applied for public read access, and a file was successfully uploaded to and downloaded from the S3 bucket.

References:

- https://aws.amazon.com/getting-started/
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/creating-bucket.html
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html

Exp No: 2	Create a Lambda Function and Configure Event Triggers
06-09-2026	

Aim:

- To create an AWS Lambda function using Python and configure an S3 event trigger.
- To test the Lambda function and verify its output using Amazon CloudWatch Logs.

Objectives:

- To understand the concept of serverless computing using AWS Lambda.
- To create an S3 bucket and use it as an event source for a Lambda function.
- To write and deploy Python code in AWS Lambda.
- To configure S3 event triggers to invoke the Lambda function automatically.
- To attach appropriate IAM permissions to the Lambda execution role.
- To verify Lambda execution through CloudWatch Logs.

Tools Used:

- Amazon Web Services (AWS) Management Console
- AWS Lambda
- Amazon S3
- Amazon CloudWatch
- AWS IAM (Identity and Access Management)
- Python 3.9 (Runtime Environment)

Theory:

AWS Lambda is a serverless computing service that lets you run code without provisioning or managing servers. Lambda automatically scales your application by running code in response to each trigger. You pay only for the compute time consumed — there is no charge when your code is not running.

An event trigger is a mechanism that automatically invokes a Lambda function in response to an event, such as a file being uploaded to an S3 bucket, a change in a DynamoDB table, or an HTTP request via API Gateway.

Key Concepts:

- Lambda Function: A piece of code that runs in response to events.
- Handler: The entry point of a Lambda function (e.g., lambda_handler in Python).
- Execution Role: An IAM role that grants the Lambda function permission to access AWS services.
- Event Source / Trigger: A service that invokes Lambda (here, Amazon S3).
- CloudWatch Logs: Used to monitor and debug Lambda function executions.
- S3 Event Notification: Automatically sends a message to Lambda when objects are created/deleted in S3.

Procedure:

Step 1: Create an S3 Bucket (if not already created)

- Go to the AWS Management Console → Open Amazon S3.
- Click Create bucket.
- Give a unique name (e.g., my-demo-upload-bucket).
- Choose a region (e.g., Asia Pacific (Mumbai)).
- Leave defaults and click Create bucket.

Step 2: Create a Lambda Function

- Open the AWS Lambda Console: https://console.aws.amazon.com/lambda
- Click Create function → Choose Author from scratch.
- Fill in:
  - Function name: logUploadedFile
  - Runtime: Python 3.9 (or latest available version)
  - Execution role: Create a new role with basic Lambda permissions
- Click Create function.

Step 3: Add Python Code to Lambda

- Scroll to the Code source section.
- Replace the default code with the following:

```
import json

def lambda_handler(event, context):
    # Extract bucket name and file name from the event
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']
    print(f"File uploaded: {key} in bucket: {bucket}")
    return {
        'statusCode': 200,
        'body': json.dumps(f"File {key} uploaded to {bucket}")
    }
```

- Click Deploy.

Step 4: Add S3 Trigger to the Lambda Function

- In the Function overview, click + Add trigger.
- Choose S3.
- Set:
  - Bucket: Choose the bucket created earlier (my-demo-upload-bucket)
  - Event type: All object create events
- Click the acknowledgment checkbox → Click Add.

Note: This grants S3 permission to invoke your Lambda function whenever a file is uploaded.

Step 5: Update Lambda Execution Role to Allow S3 Read Access

- Go to the Configuration tab → Permissions.
- Click on the Role name → This opens the IAM console.
- Click Add permissions → Attach policies.
- Search for and select: AmazonS3ReadOnlyAccess.
- Click Next → Add permissions.

Step 6: Test the Lambda Function

- Open the Amazon S3 Console.
- Go to your bucket (my-demo-upload-bucket).
- Click Upload → Add any small file (e.g., sample.txt) → Click Upload.

Step 7: View the Output in CloudWatch Logs

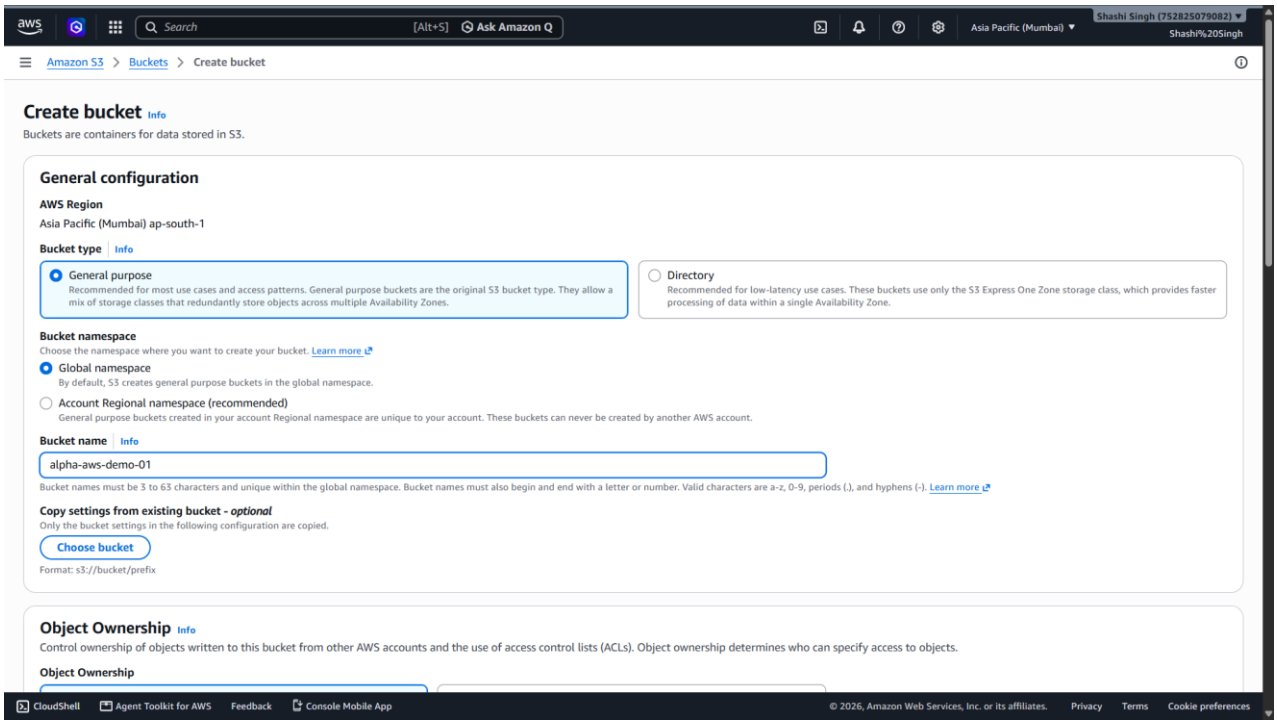
- Go back to the Lambda function.
- Click Monitor tab → View logs in CloudWatch.
- Click the latest log stream.
- We see logs similar to:

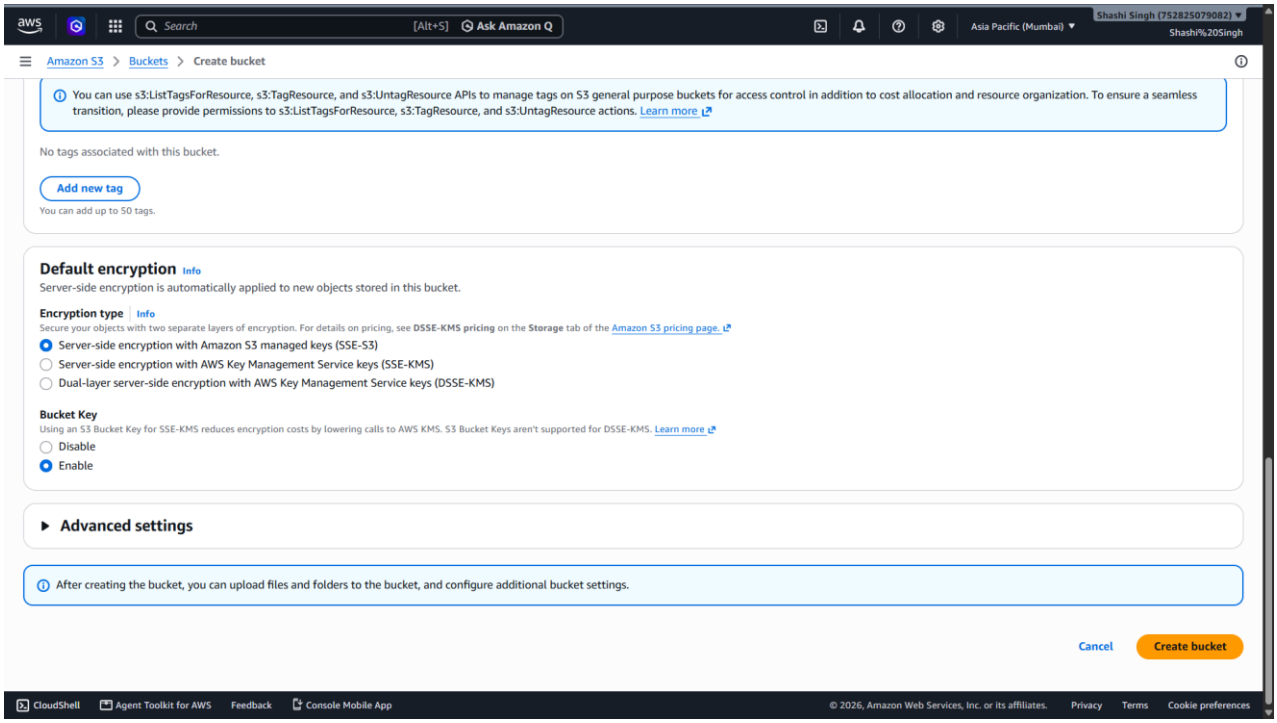
START RequestId: xxx
File uploaded: sample.txt in bucket: my-demo-upload-bucket
END RequestId: xxx

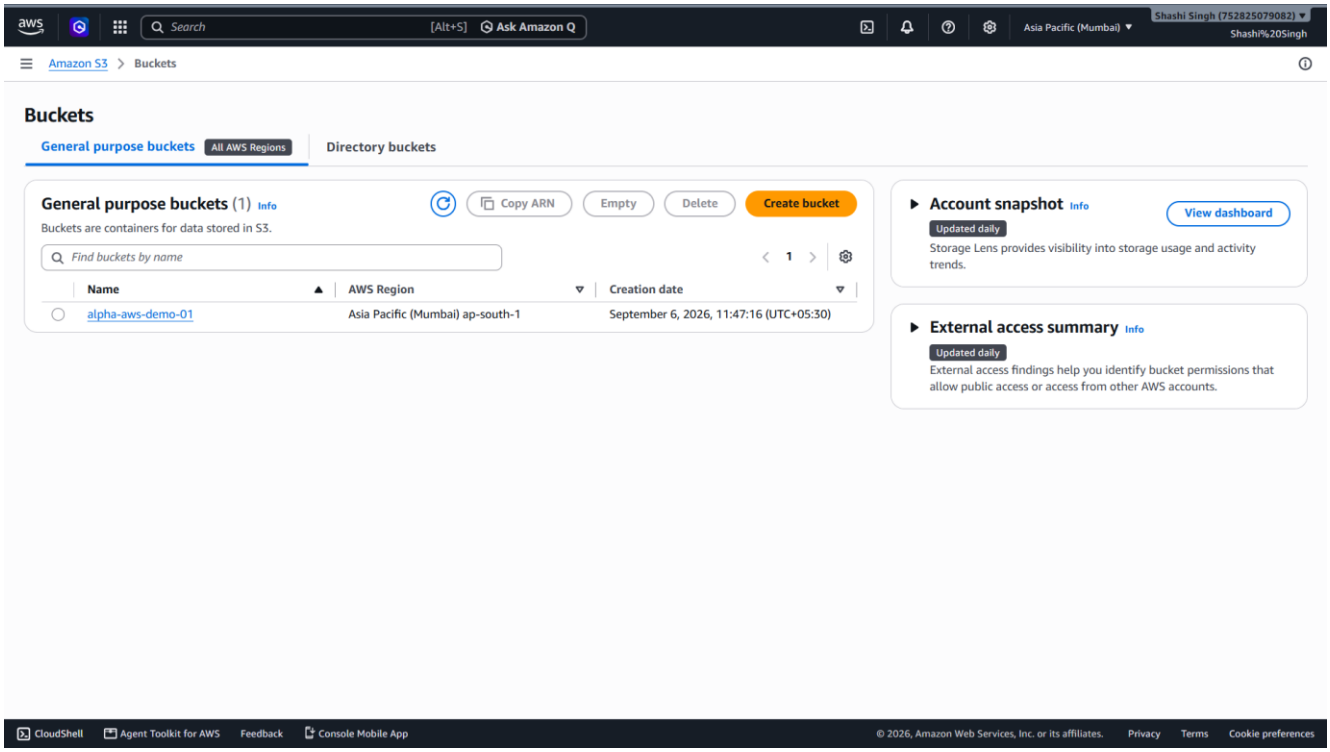
This confirms that the Lambda function was successfully triggered and executed.

Output:

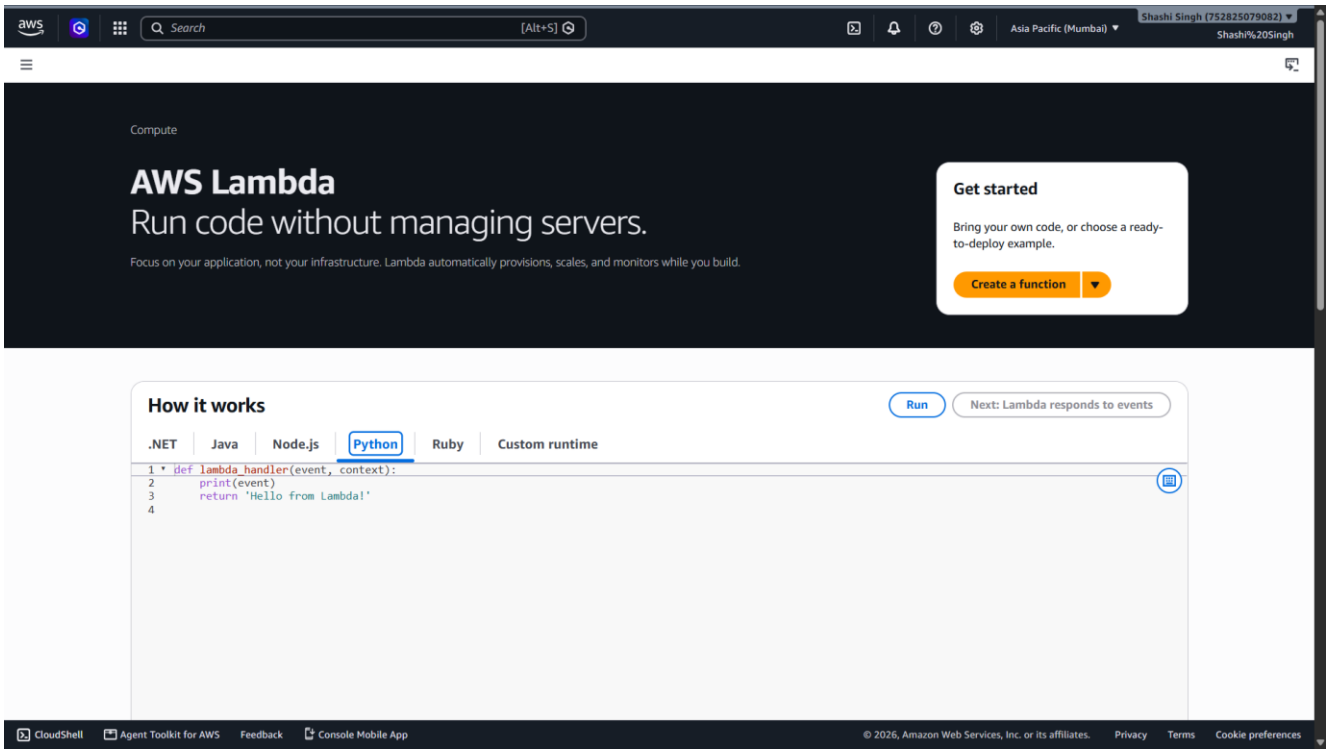
- Screenshot 1: S3 Bucket created (alpha-aws-demo-01)

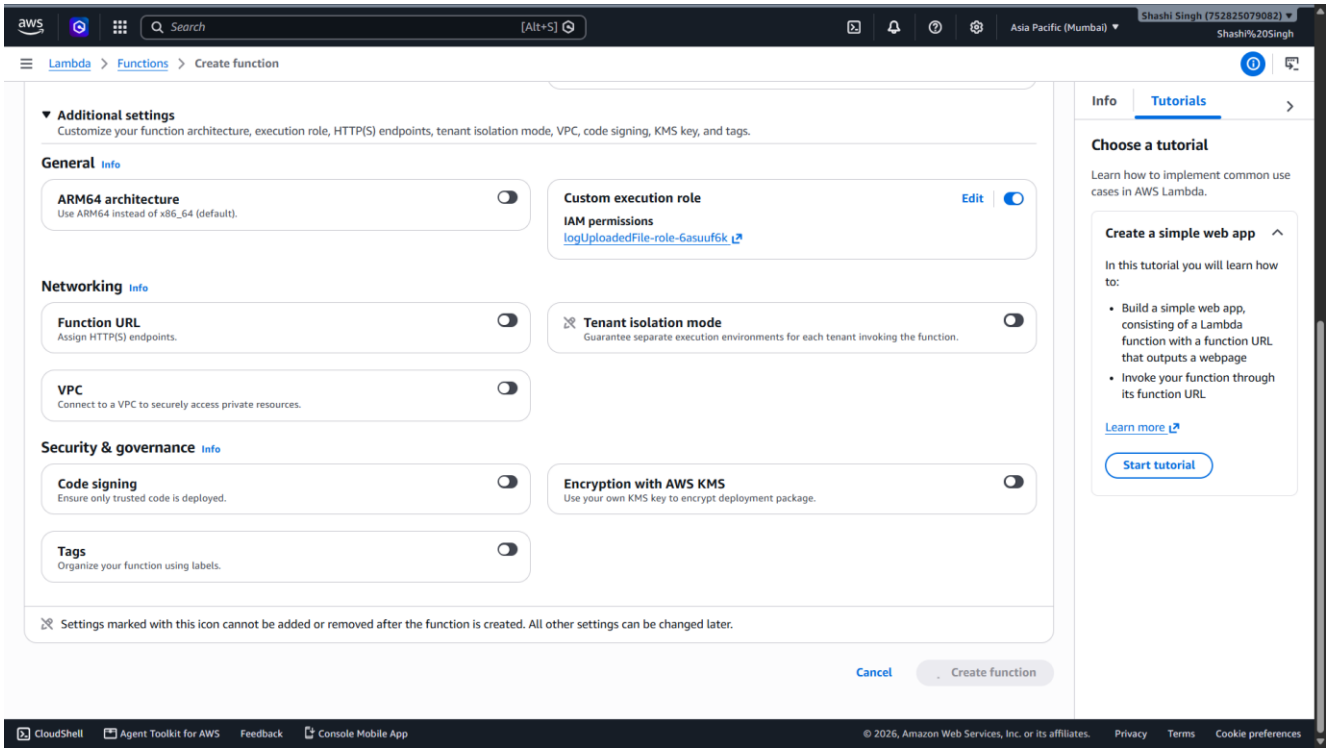




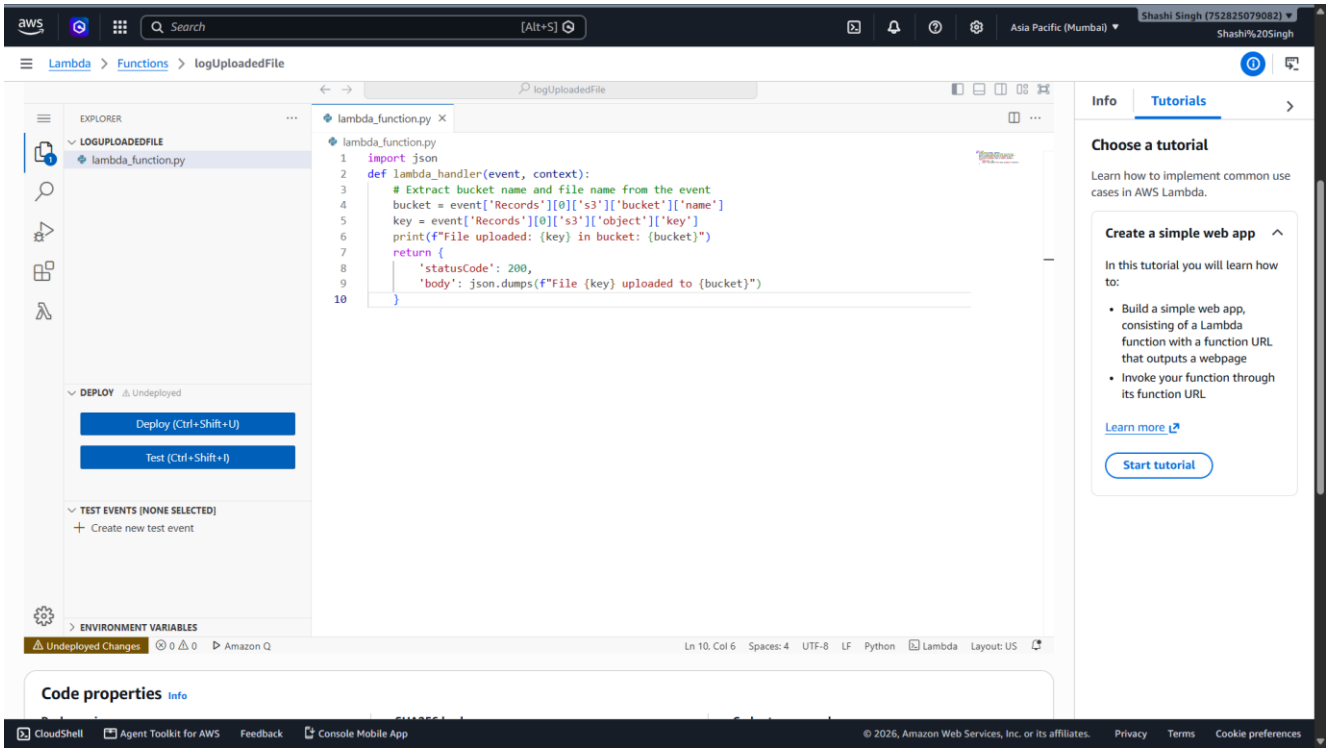


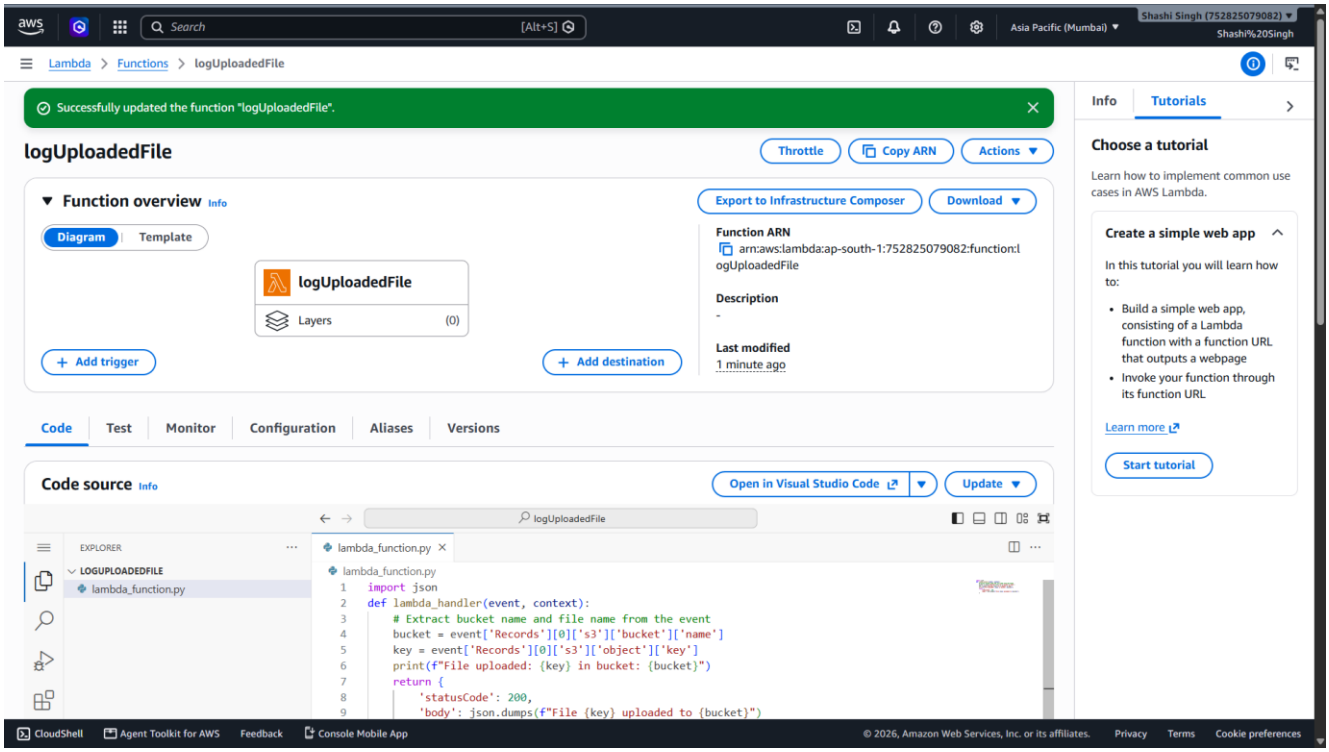
- Screenshot 2: Lambda function created (logUploadedFile) with Python 3.9 runtime



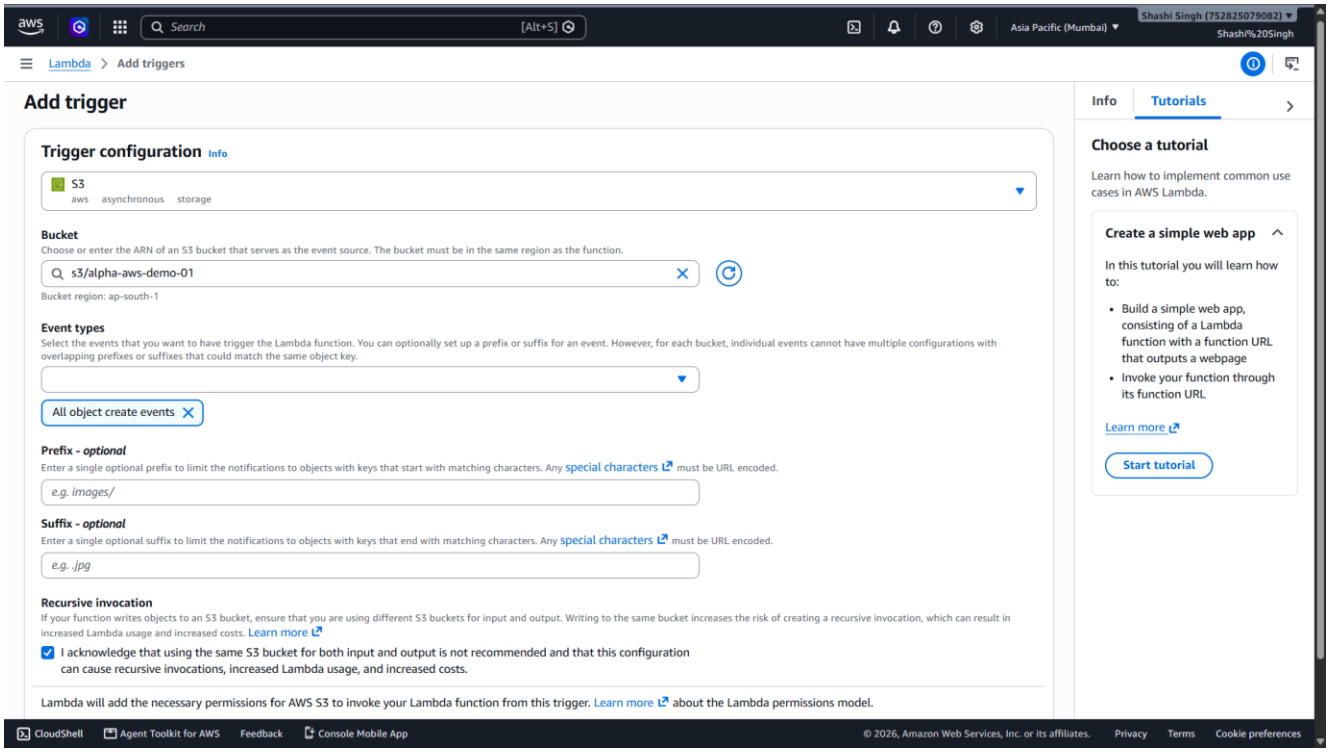


- Screenshot 3: Python code deployed in Lambda Code source editor

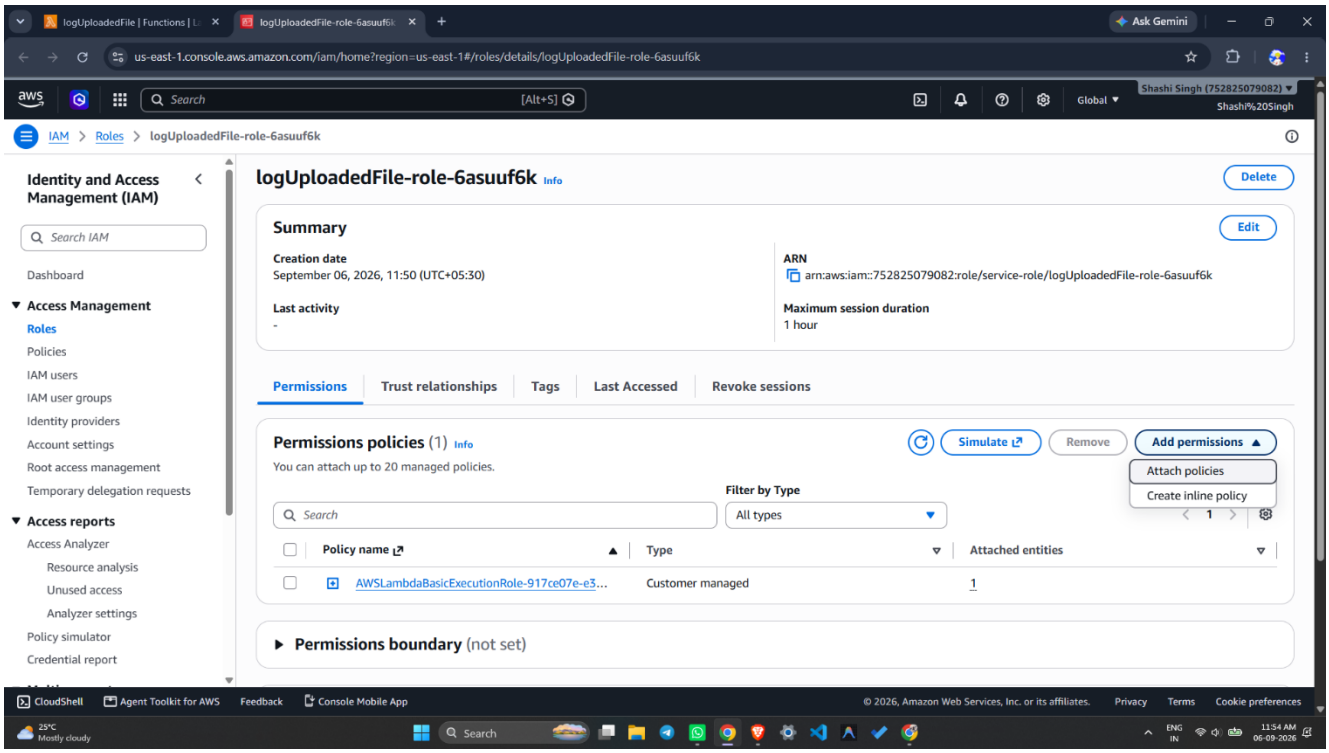


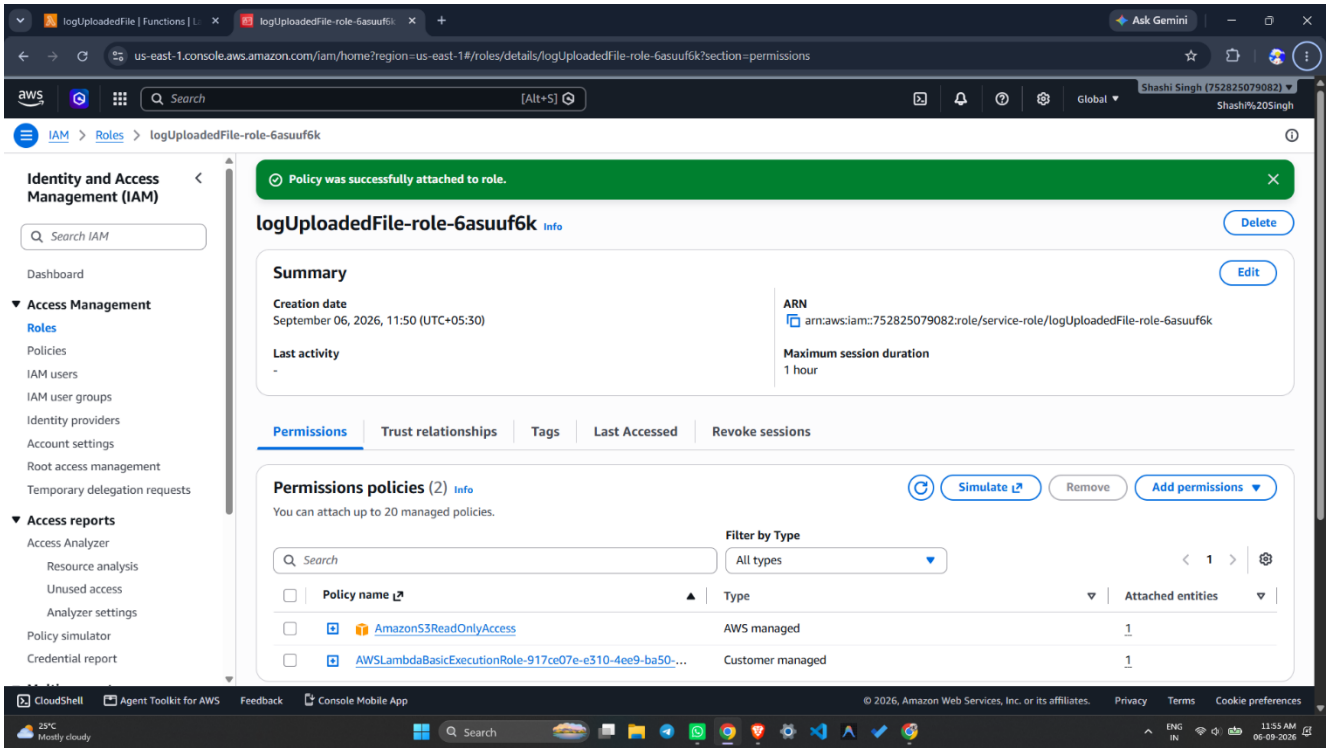


- Screenshot 4: S3 trigger added in Function overview

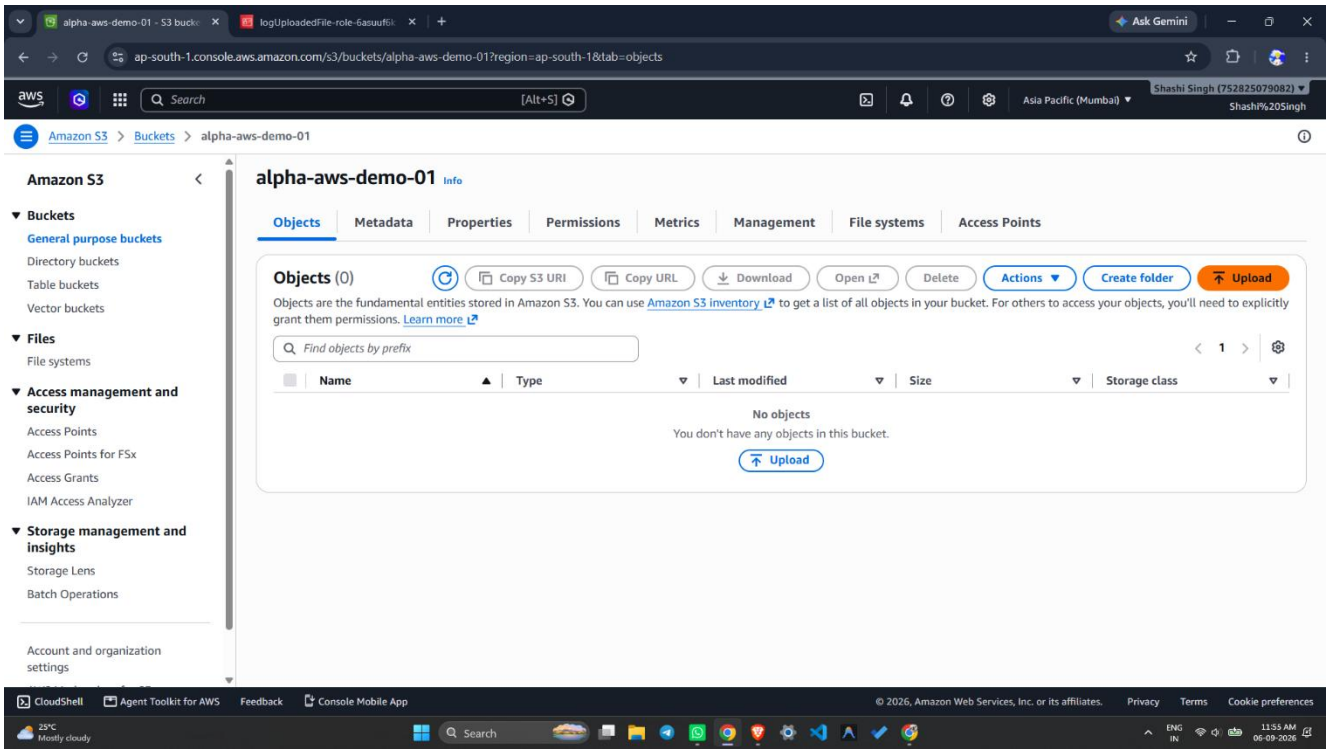


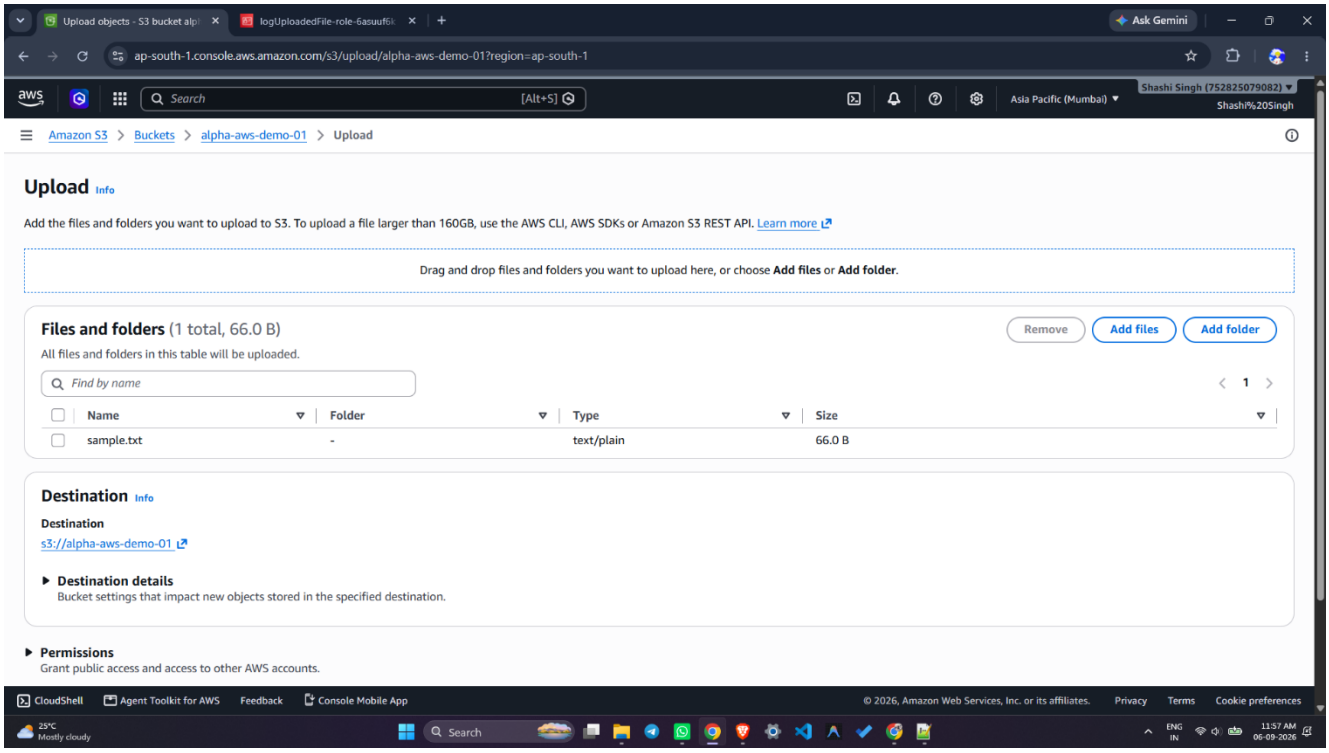
- Screenshot 5: AmazonS3ReadOnlyAccess policy attached to execution role (IAM)

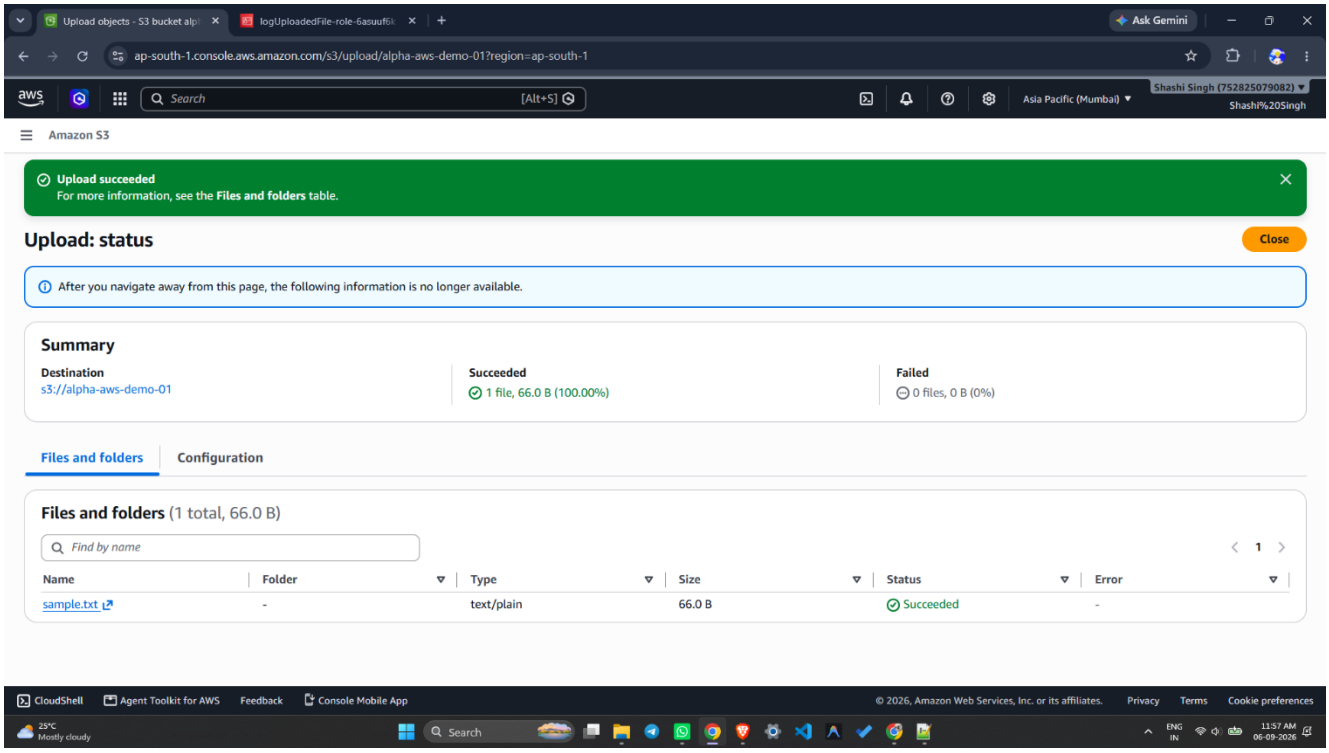




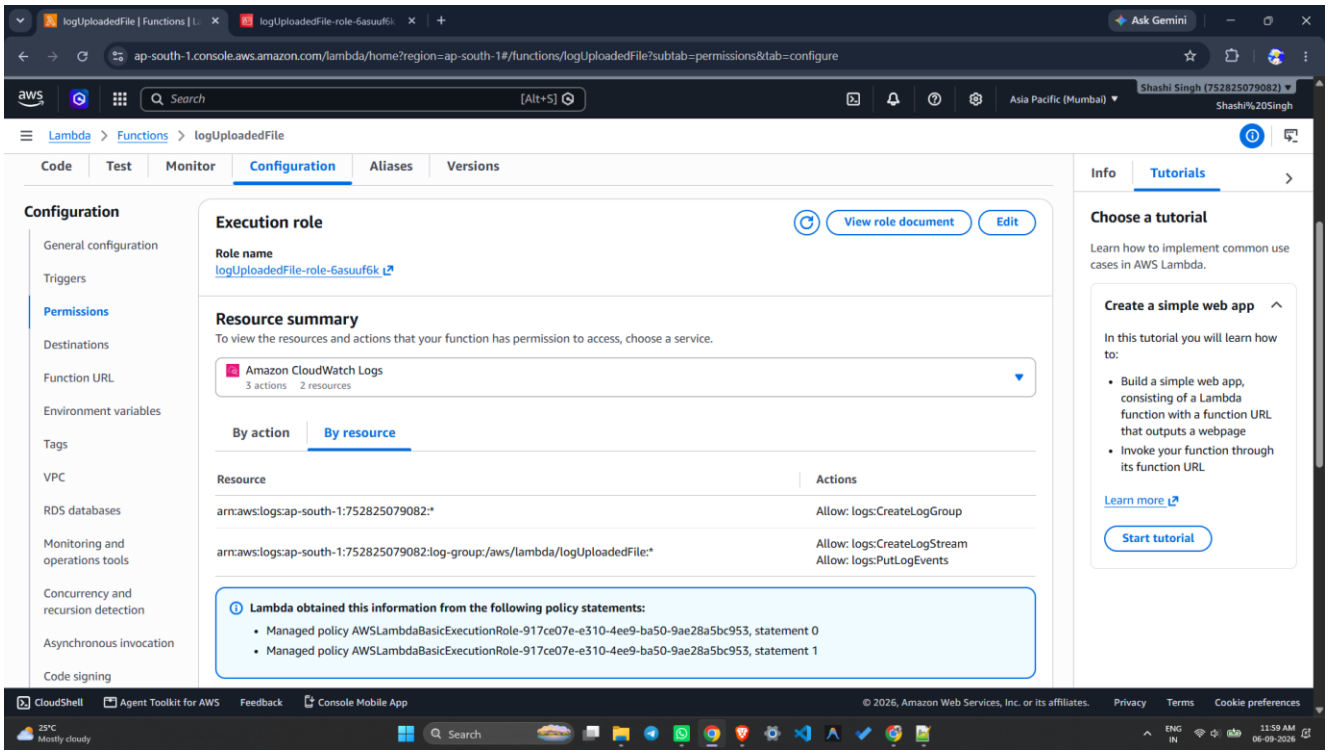
- Screenshot 6: File (sample.txt) uploaded to S3 bucket







- Screenshot 7: CloudWatch Logs showing "File uploaded: sample.txt in bucket: my-demo-upload-bucket"



Result:

Successfully created an AWS Lambda function using Python 3.9 with the name logUploadedFile. An S3 event trigger was configured to automatically invoke the Lambda function upon file upload. The Lambda execution role was updated with AmazonS3ReadOnlyAccess permissions. Upon uploading a file (sample.txt) to the S3 bucket, the Lambda function was triggered automatically, and the output was verified in Amazon CloudWatch Logs.

References:

- https://docs.aws.amazon.com/lambda/latest/dg/getting-started.html
- https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html
- https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/Working-with-log-groups-and-streams.html
- https://aws.amazon.com/lambda/

Exp No: 3	Identity and Access Management (IAM) in AWS
06-09-2026	

Aim:

- To create and manage AWS Identity and Access Management (IAM) users, groups, roles, and policies.
- To manage access permissions to AWS resources and verify them using the IAM Policy Simulator.

Objectives:

- To understand the concept of IAM and its components (users, groups, roles, policies).
- To create an IAM user and configure console access.
- To create an IAM group and attach policies to it.
- To add users to groups and verify inherited permissions.
- To create a custom IAM policy using JSON.
- To create an IAM role for AWS services.
- To test and verify permissions using the IAM Policy Simulator.

Tools Used:

- Amazon Web Services (AWS) Management Console
- AWS IAM (Identity and Access Management)
- Amazon S3 (for testing permissions)
- IAM Policy Simulator — https://policysim.aws.amazon.com
- Web Browser (Chrome / Firefox)

Theory:

AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. IAM allows you to manage who is authenticated (signed in) and authorized (has permissions) to use AWS resources.

Key IAM Components:

- User: An entity that represents a person or application that can interact with AWS. Each user has unique credentials.
- Group: A collection of IAM users. Permissions assigned to a group are inherited by all users in it.
- Role: An IAM identity with specific permissions that can be assumed by AWS services or users. Roles are not associated with a specific person.
- Policy: A JSON document that defines what actions are allowed or denied on which resources. Policies are attached to users, groups, or roles.

Types of Policies:

- AWS Managed Policies: Pre-built by AWS (e.g., AmazonS3ReadOnlyAccess).
- Customer Managed Policies: Custom policies created by the user.
- Inline Policies: Policies embedded directly into a user, group, or role.

Procedure:

Step 1: Log in to AWS Console

- Go to AWS Management Console and sign in using the root account.
- The root account has full IAM access by default.

Step 2: Create an IAM User

- Go to Services → IAM → Users → Add users.
- Enter User name: e.g., Alice.
- Select Access type:
  - Programmatic access — for AWS CLI or SDK.
  - AWS Management Console access — allows browser login (select this for testing).
- Set a Console password (Autogenerated or Custom).
- Enable Require password reset if needed.
- Click Next: Permissions → choose Attach existing policies directly.
- Select AmazonS3ReadOnlyAccess (limited access — user can only read S3).
- Click Next: Tags (optional) → Next: Review → Create user.
- Download the .csv file with login credentials if needed.

Step 3: Create an IAM Group

- Go to IAM → Groups → Create group.
- Enter Group name: e.g., Developers.
- Attach policy: AmazonS3ReadOnlyAccess.
- Click Create group.

Step 4: Add Users to the Group

- Go to Users → Alice → Add to Group.
- Select Developers group → click Add.
- Alice now inherits all permissions from the Developers group.

Step 5: Create IAM Policy (Optional)

- Go to IAM → Policies → Create policy.
- Select the JSON tab and enter the following:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

```
        "Action": ["s3:GetObject", "s3:ListBucket"],
        "Resource": [
          "arn:aws:s3:::my-bucket",
          "arn:aws:s3:::my-bucket/*"
        ]
      }
    ]
}
```

- Click Review policy → Name it S3ReadOnlyPolicy → Create policy.
- Attach it to the desired User or Group.

Step 6: Create IAM Role (Optional for Services)

- Go to IAM → Roles → Create role.
- Select AWS service → e.g., EC2.
- Attach policy → e.g., AmazonS3FullAccess.
- Name the role → e.g., EC2S3AccessRole.
- Click Create role.

Step 7: Test IAM User Access & Use IAM Policy Simulator

Step A: Prepare test environment (as Root User)

- Log in as Root User.
- Go to S3 → Create an S3 bucket and upload a sample file (test.txt).
- Log out from Root User.

Step B: Login as IAM User (Alice)

- Go to AWS Management Console → IAM user sign-in.
- Enter Account ID / alias (from root account → IAM dashboard).
- IAM user name: Alice | Password: (set during creation).
- Click Sign in.
- Verify that Alice can list/read S3 but cannot write or delete.

Step C: Use IAM Policy Simulator

- Log in as Root User.
- Go to Services → IAM → Policy Simulator (left side menu).
- Direct link: https://policysim.aws.amazon.com
- Select the IAM Identity to test (User: Alice, Group: Developers, or Role: EC2S3AccessRole).
- This loads all attached policies for that identity.

Choose a Service and Action:

- Select AWS service: Amazon S3.
- Select Actions: ListBucket, GetObject, PutObject.

Run the Simulation:

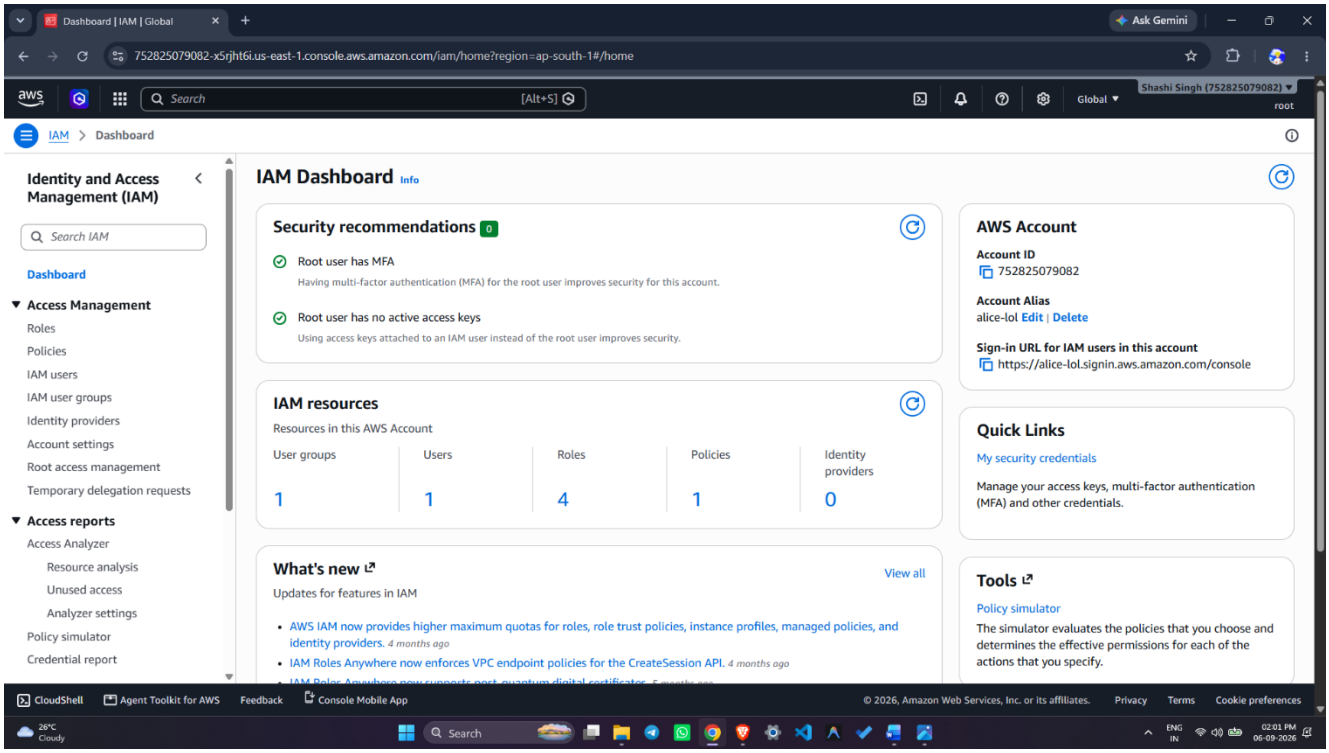
- Click Run Simulation.
- Results will show whether each action is Allowed ✅ or Denied ❌.

Interpret Results:

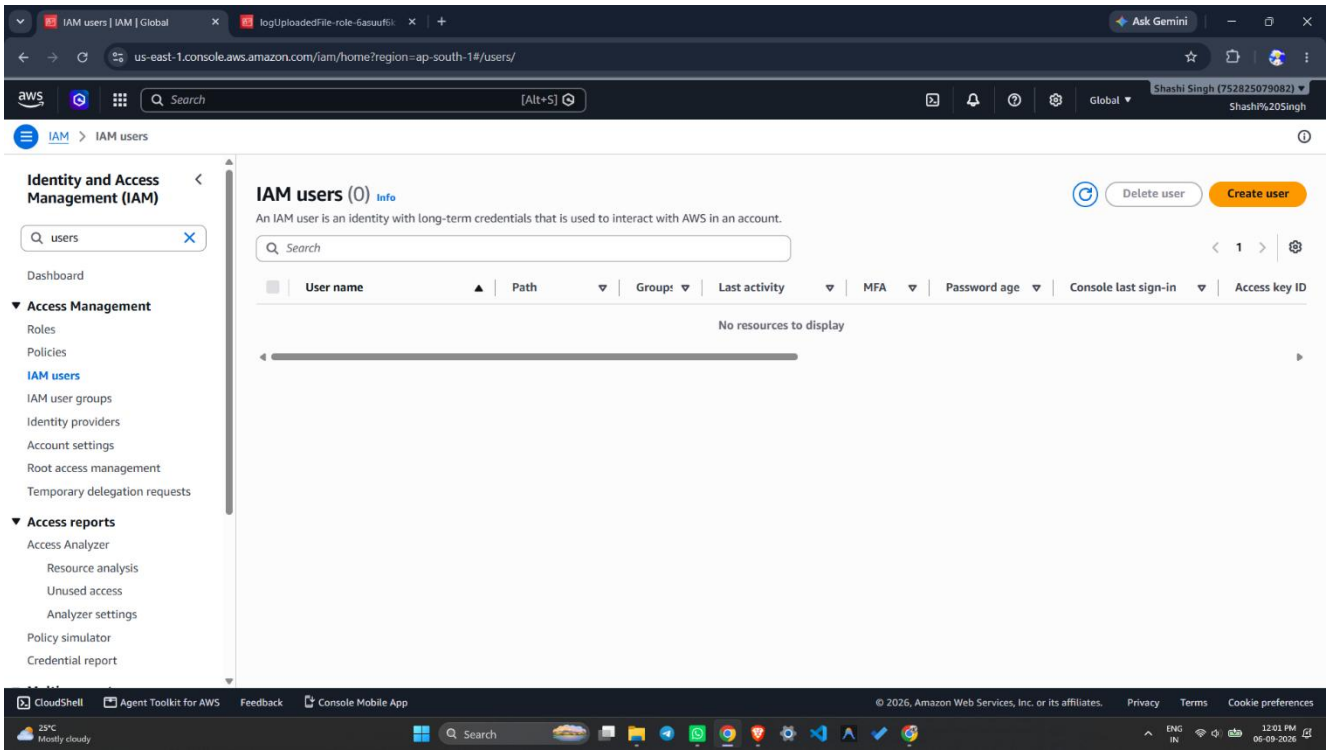
- If Allowed → The policy grants that permission.
- If Denied → The policy restricts it, or an explicit deny exists.

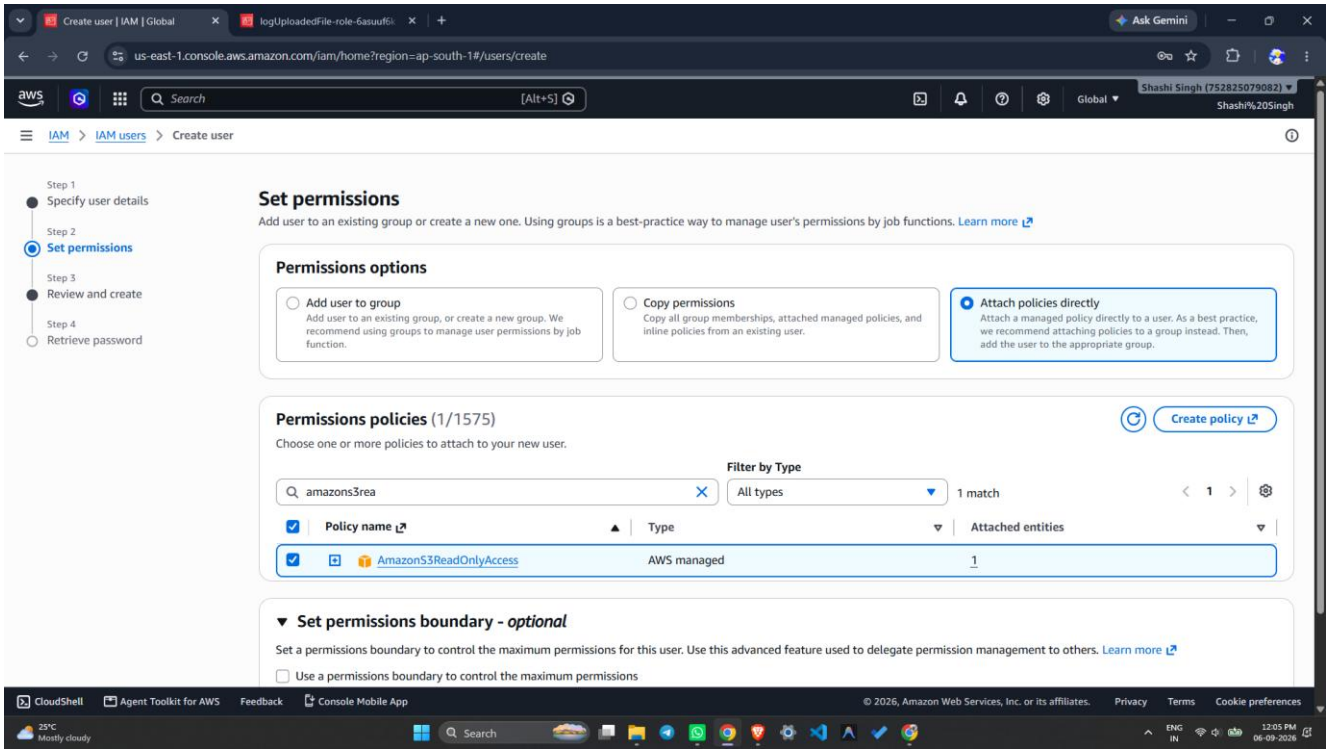
Output:

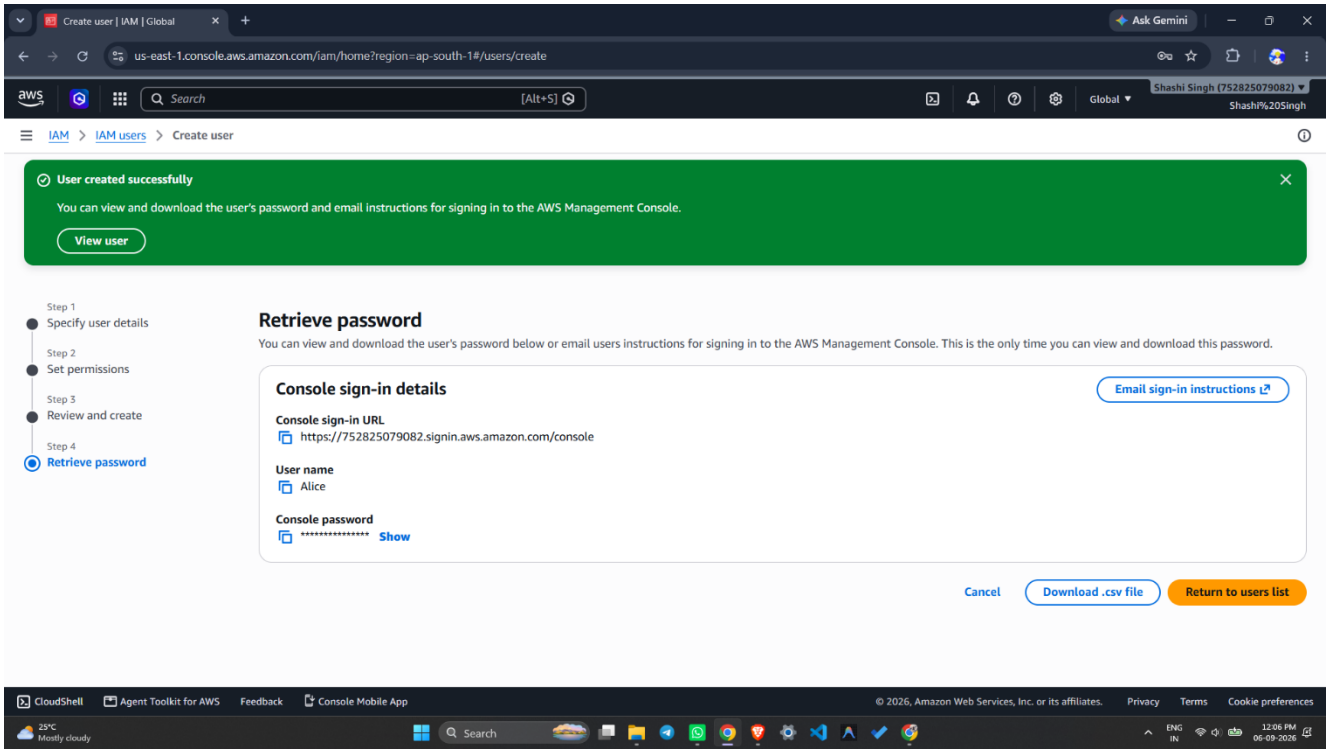
- Screenshot 1: IAM Dashboard overview



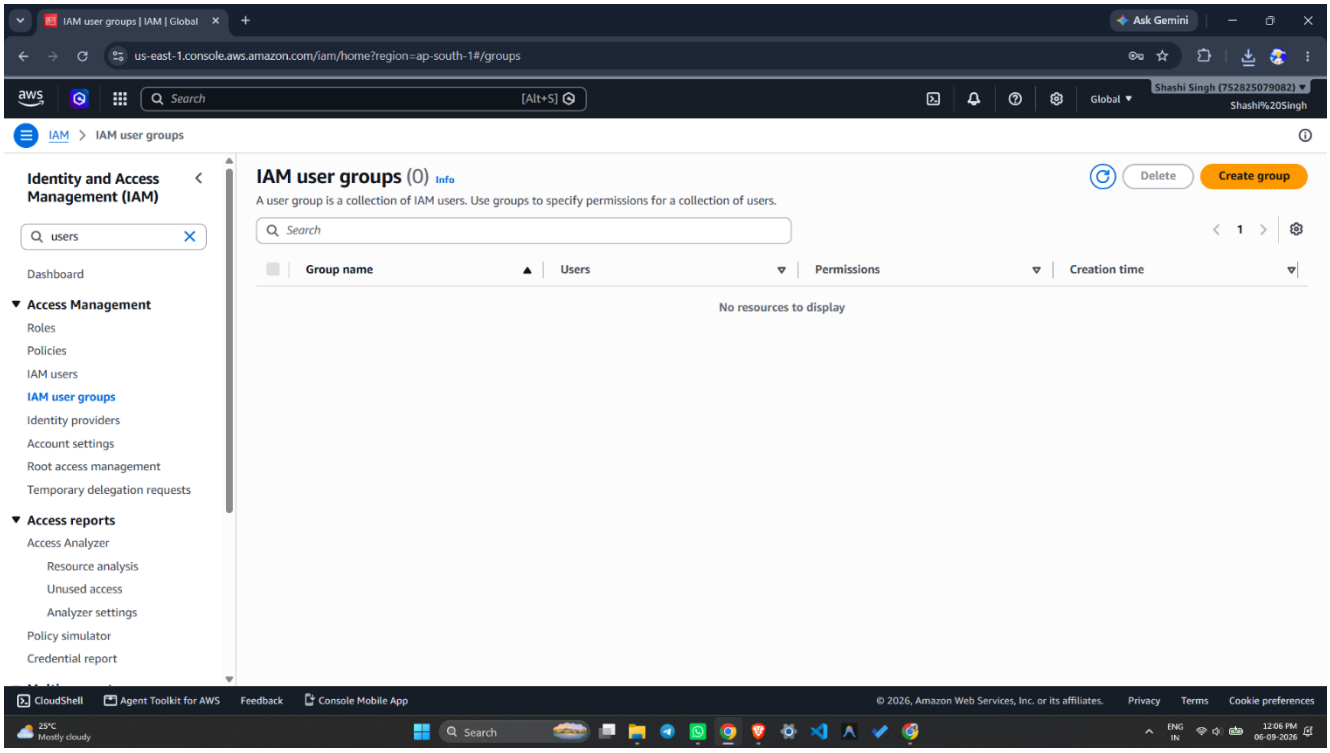
- Screenshot 2: IAM User "Alice" created with AmazonS3ReadOnlyAccess policy

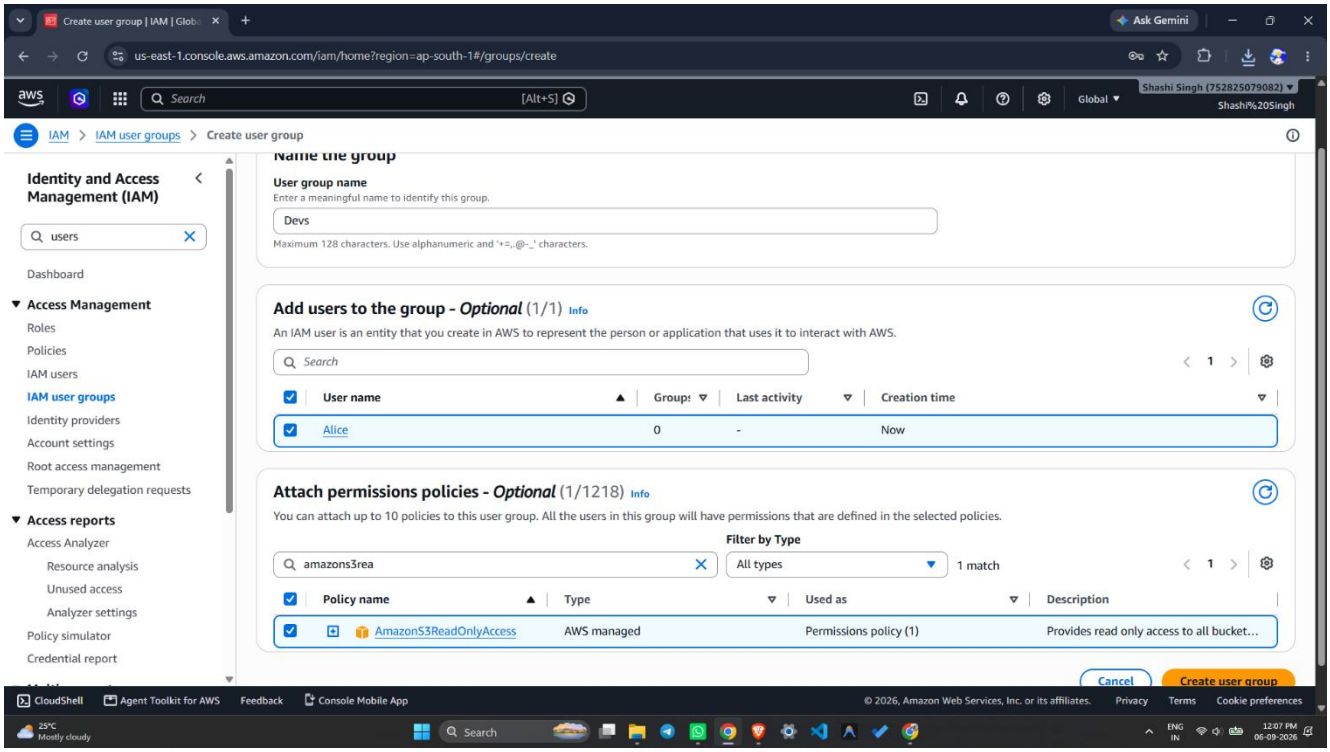


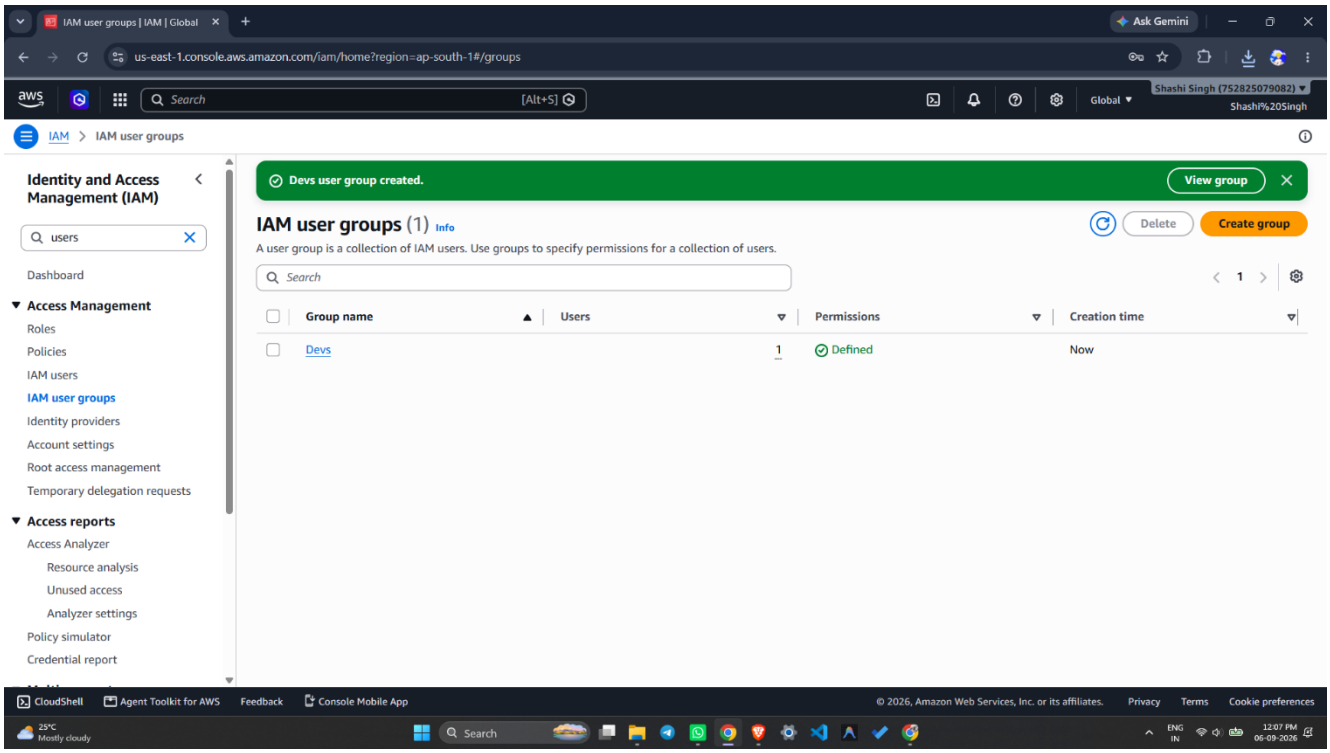




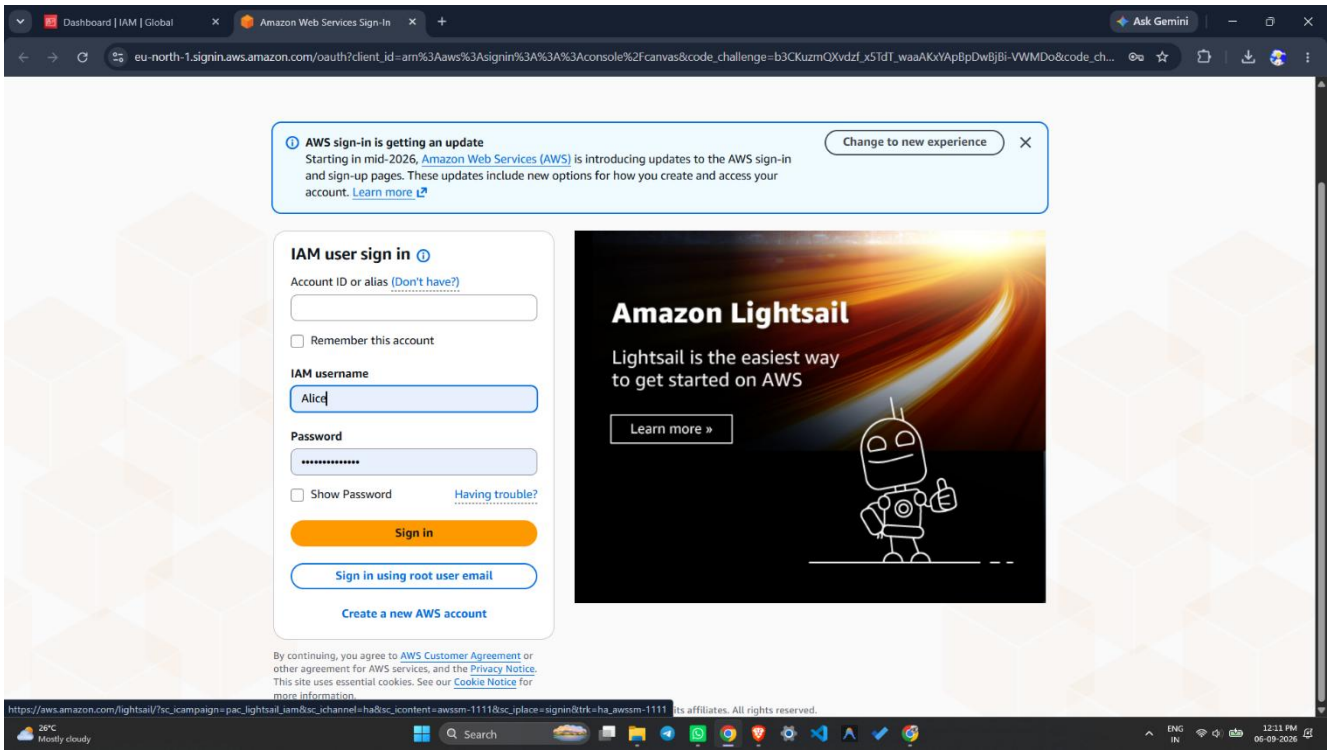
- Screenshot 3: IAM Group "Developers" created with policy attached

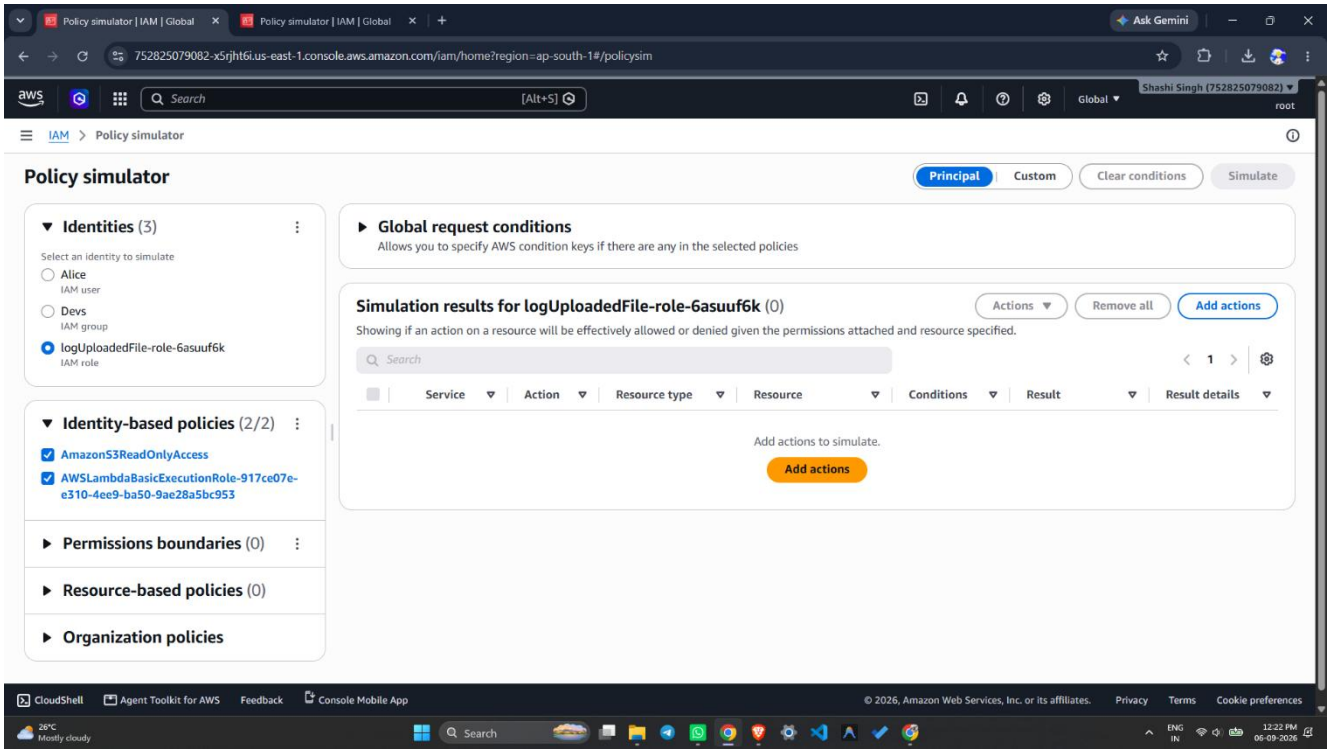


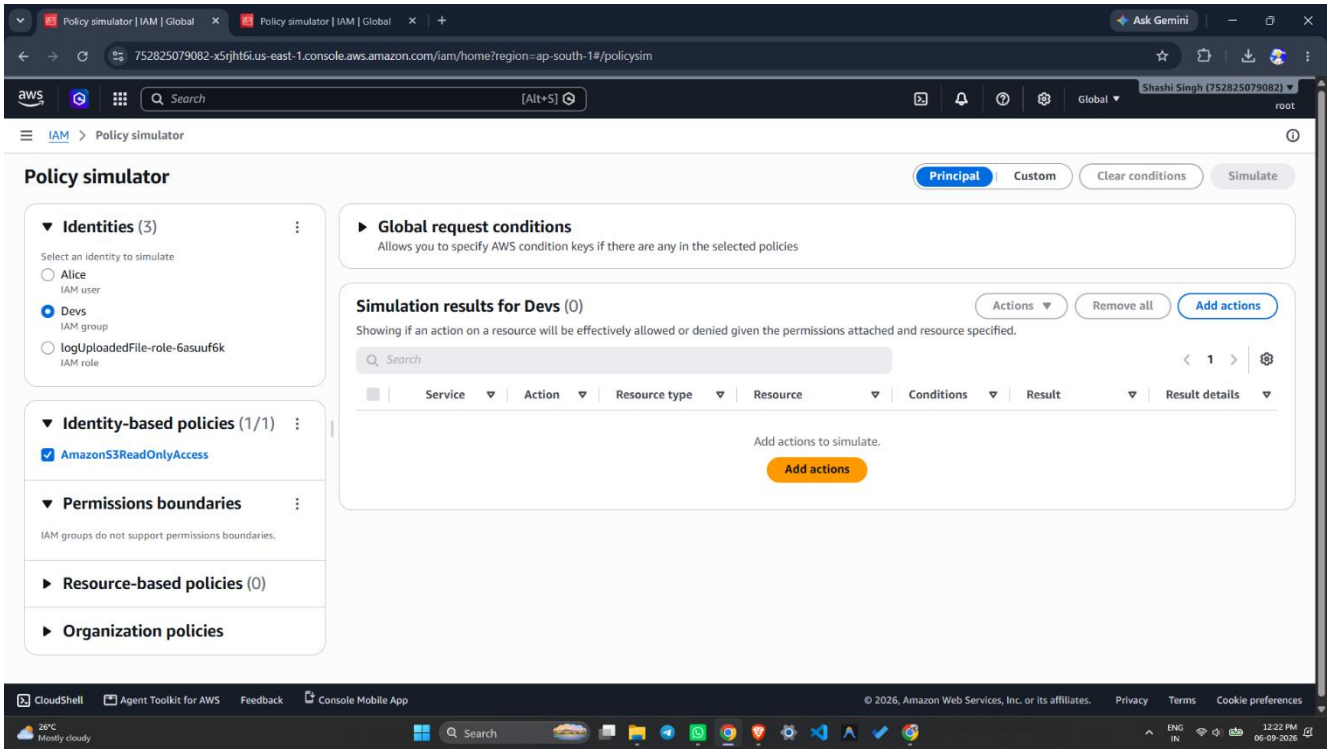

Create user group | IAM | Global
Ask Gemini
us-east-1.console.aws.amazon.com/iam/home?region=ap-south-1#/groups/create
Search
[Alt+S]
Global
Shashi Singh (752825079082)
Shashi%20Singh
IAM > IAM user groups > Create user group
Identity and Access Management (IAM)
users
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
Name the group
User group name
Enter a meaningful name to identify this group.
Devs
Maximum 128 characters. Use alphanumeric and '+=,.@-_' characters.
Add users to the group - Optional (1/1) Info
An IAM user is an entity that you create in AWS to represent the person or application that uses it to interact with AWS.
Search
User name
Groups
Last activity
Creation time
Alice
0
-
Now
Attach permissions policies - Optional (1/1218) Info
You can attach up to 10 policies to this user group. All the users in this group will have permissions that are defined in the selected policies.
Filter by Type
amazons3rea
All types
1 match
Policy name
Type
Used as
Description
AmazonS3ReadOnlyAccess
AWS managed
Permissions policy (1)
Provides read only access to all bucket...
Cancel
Create user group
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
25°C
Mostly cloudy
Search
ENG
IN
12:07 PM
06-09-2026

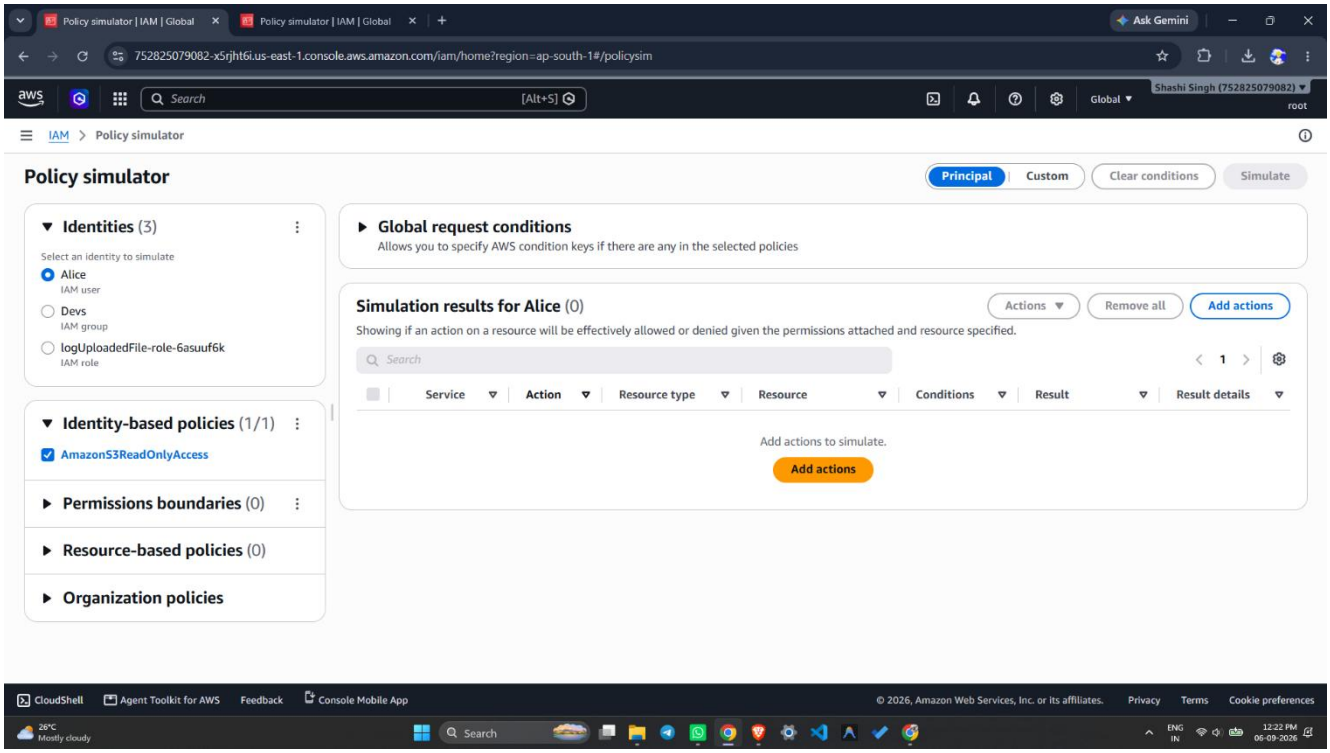

IAM user groups | IAM | Global
Ask Gemini
us-east-1.console.aws.amazon.com/iam/home?region=ap-south-1#/groups
Search
[Alt+S]
Global
Shashi Singh (752825079082)
Shashi%20Singh
IAM > IAM user groups
Identity and Access Management (IAM)
users
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
Devs user group created.
View group
IAM user groups (1) Info
A user group is a collection of IAM users. Use groups to specify permissions for a collection of users.
Delete
Create group
Search
Group name
Users
Permissions
Creation time
Devs
1
Defined
Now
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
25°C
Mostly cloudy
Search
ENG
IN
12:07 PM
06-09-2026

- Screenshot 4: IAM Policy Simulator — ListBucket: Allowed, PutObject: Denied






Policy simulator | IAM | Global
Policy simulator | IAM | Global
Ask Gemini
752825079082-x5rjht6i.us-east-1.console.aws.amazon.com/iam/home?region=ap-south-1#/policysim
aws
Search
[Alt+S]
Global
Shashi Singh (752825079082)
root
IAM > Policy simulator
Policy simulator
Principal
Custom
Clear conditions
Simulate
Identities (3)
Select an identity to simulate
Alice
IAM user
Devs
IAM group
logUploadedFile-role-6asuuf6k
IAM role
Identity-based policies (1/1)
AmazonS3ReadOnlyAccess
Permissions boundaries
IAM groups do not support permissions boundaries.
Resource-based policies (0)
Organization policies
Global request conditions
Allows you to specify AWS condition keys if there are any in the selected policies
Simulation results for Devs (0)
Actions
Remove all
Add actions
Showing if an action on a resource will be effectively allowed or denied given the permissions attached and resource specified.
Search
Service
Action
Resource type
Resource
Conditions
Result
Result details
Add actions to simulate.
Add actions
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
26°C
Mostly cloudy
Search
ENG
IN
12:22 PM
06-09-2026


Policy simulator | IAM | Global
Policy simulator | IAM | Global
Ask Gemini
752825079082-x5rjht6i.us-east-1.console.aws.amazon.com/iam/home?region=ap-south-1#/policysim
aws
Search
[Alt+S]
Global
Shashi Singh (752825079082)
root
IAM > Policy simulator
Policy simulator
Principal
Custom
Clear conditions
Simulate
Identities (3)
Select an identity to simulate
Alice
IAM user
Devs
IAM group
logUploadedFile-role-6asuuf6k
IAM role
Identity-based policies (1/1)
AmazonS3ReadOnlyAccess
Permissions boundaries (0)
Resource-based policies (0)
Organization policies
Global request conditions
Allows you to specify AWS condition keys if there are any in the selected policies
Simulation results for Alice (0)
Actions
Remove all
Add actions
Showing if an action on a resource will be effectively allowed or denied given the permissions attached and resource specified.
Search
Service
Action
Resource type
Resource
Conditions
Result
Result details
Add actions to simulate.
Add actions
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
26°C
Mostly cloudy
Search
ENG
IN
12:22 PM
06-09-2026

Result:

Successfully created IAM users, groups, roles, and policies in AWS. The IAM user "Alice" was created and assigned AmazonS3ReadOnlyAccess, and was added to the "Developers" group. A custom IAM policy S3ReadOnlyPolicy was created using JSON and attached to the user/group. An IAM role EC2S3AccessRole was created for the EC2 service. Permissions were verified using the IAM Policy Simulator, which confirmed that ListBucket and GetObject were Allowed while PutObject was Denied for the IAM user Alice.

References:

- https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html
- https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html
- https://policysim.aws.amazon.com
- https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html